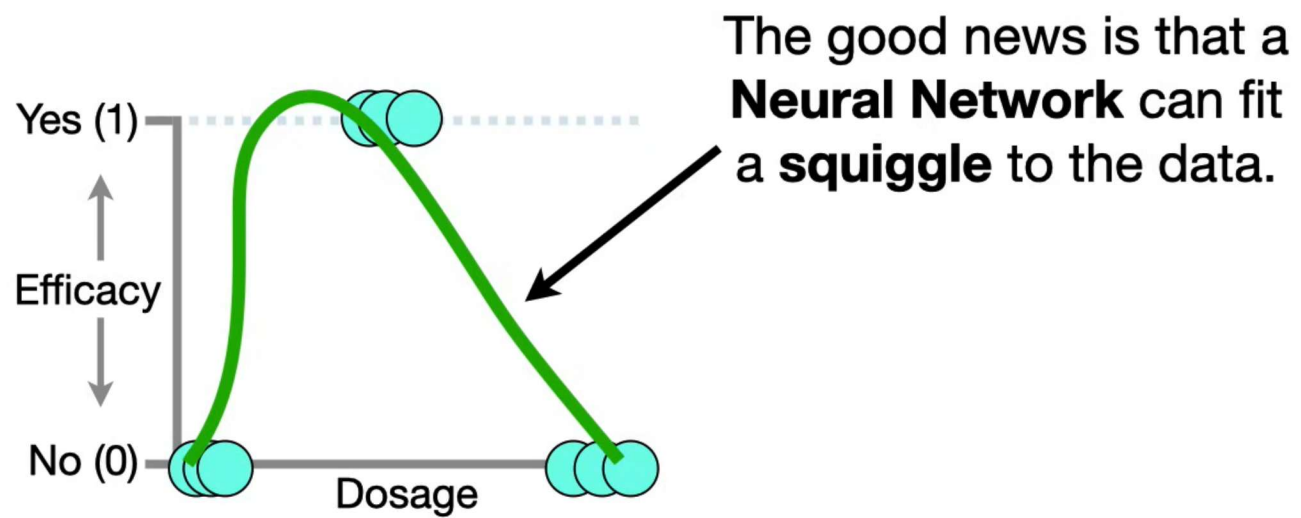
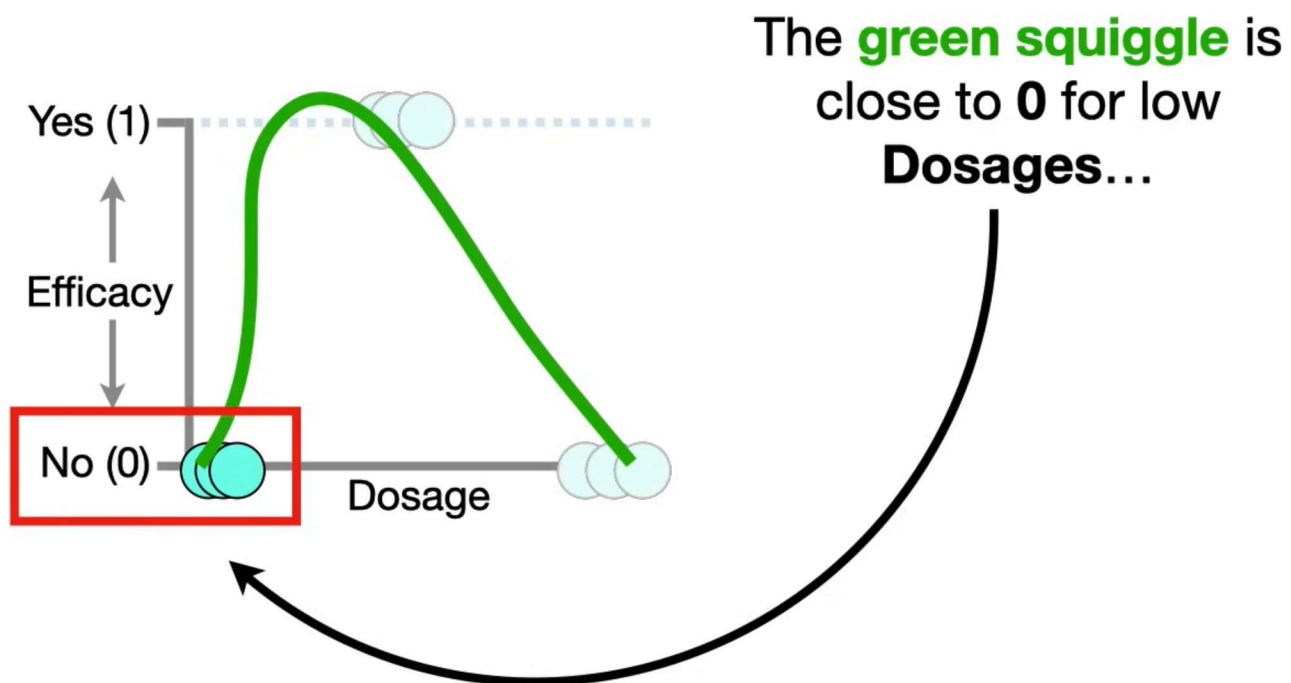
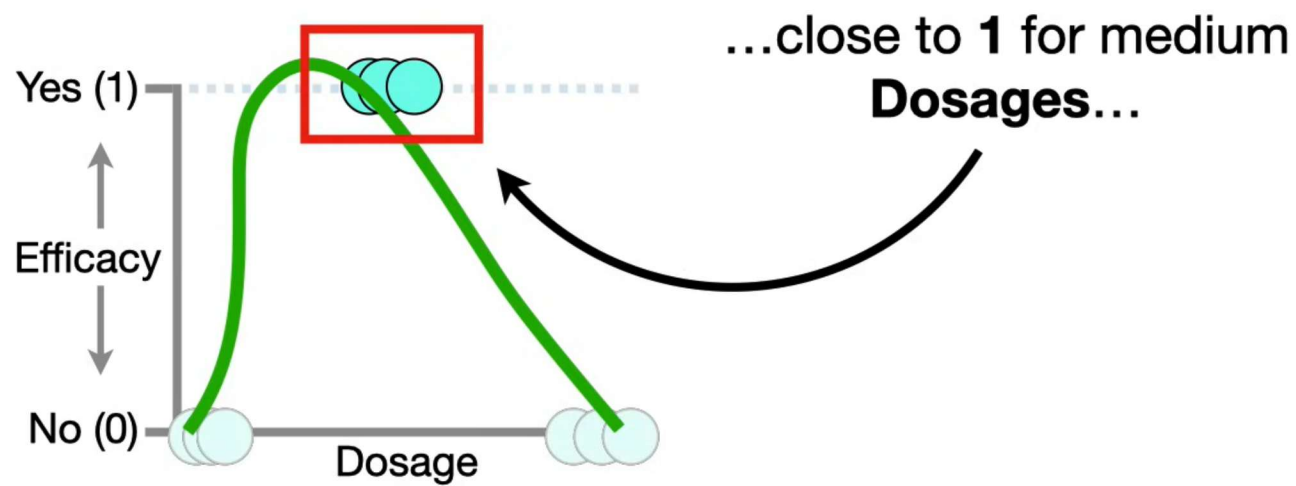
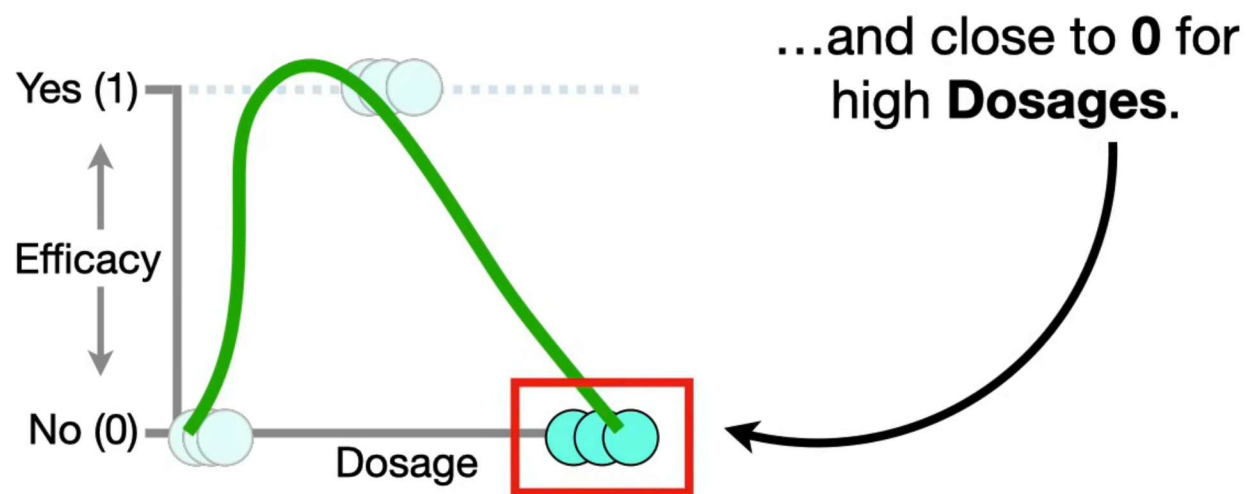
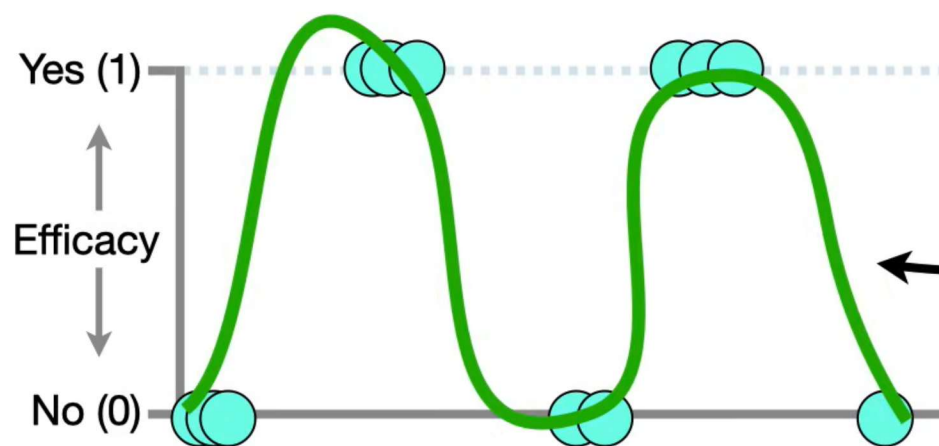












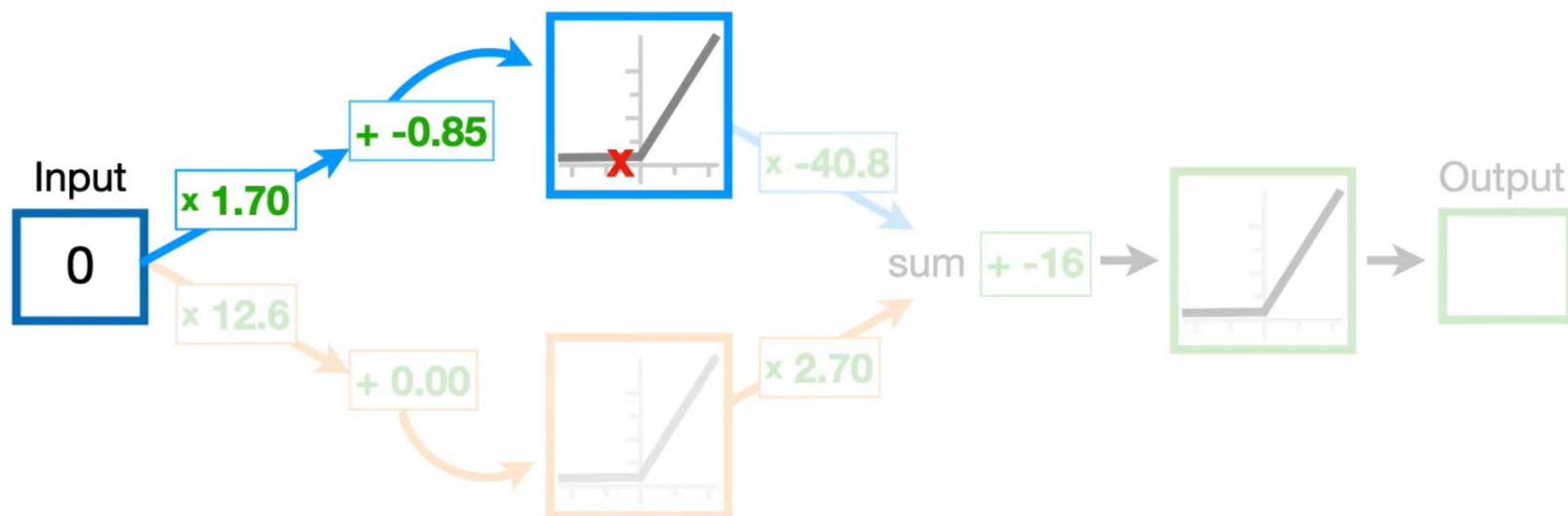
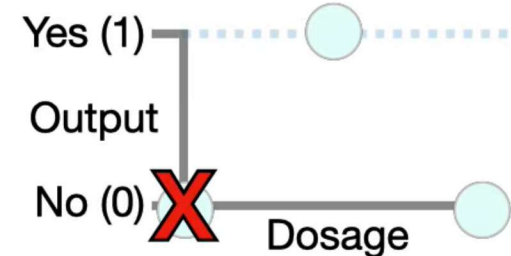
...a **Neural Network**
can fit a **squiggle** to it.



Because $0 > -0.85$, the output from the **ReLU** activation function is **0**...

$$(0 \times 1.70) + -0.85 = -0.85$$

$$f(-0.85) = \max(0, -0.85) = 0$$

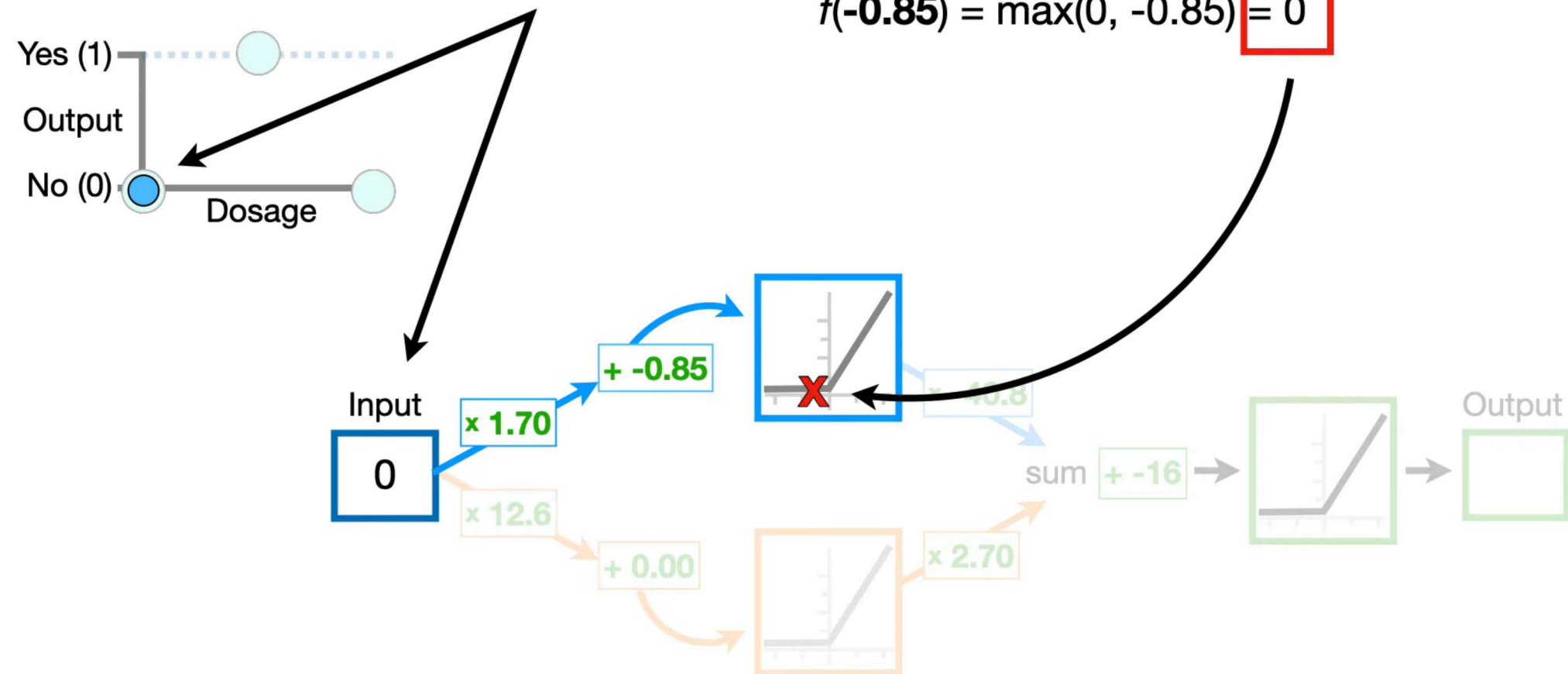




So let's put a **blue dot** at **0** for when **Dosage = 0**.

$$(0 \times 1.70) + -0.85 = -0.85$$

$$f(-0.85) = \max(0, -0.85) = 0$$

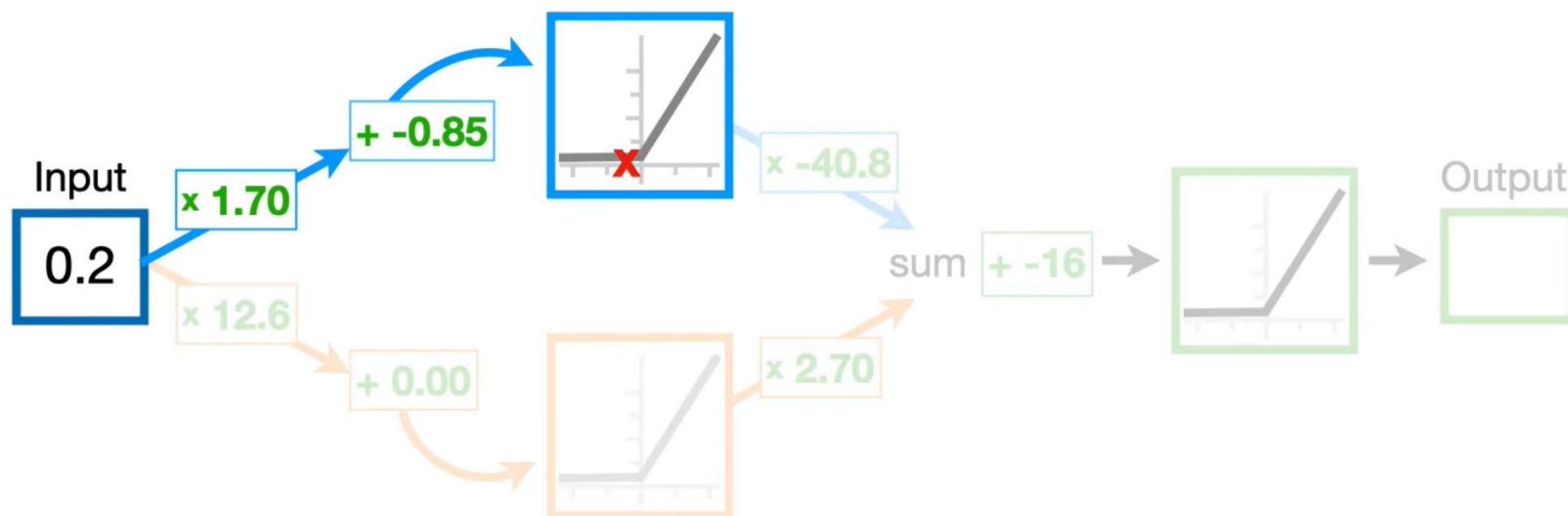
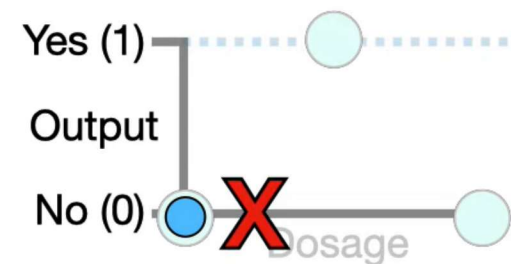




$$(0.1 \times 1.70) + -0.85 = -0.51$$

...and again, because $0 > -0.51$, the output from the **ReLU** activation function is **0**.

$$f(-0.51) = \max(0, -0.51) = 0$$

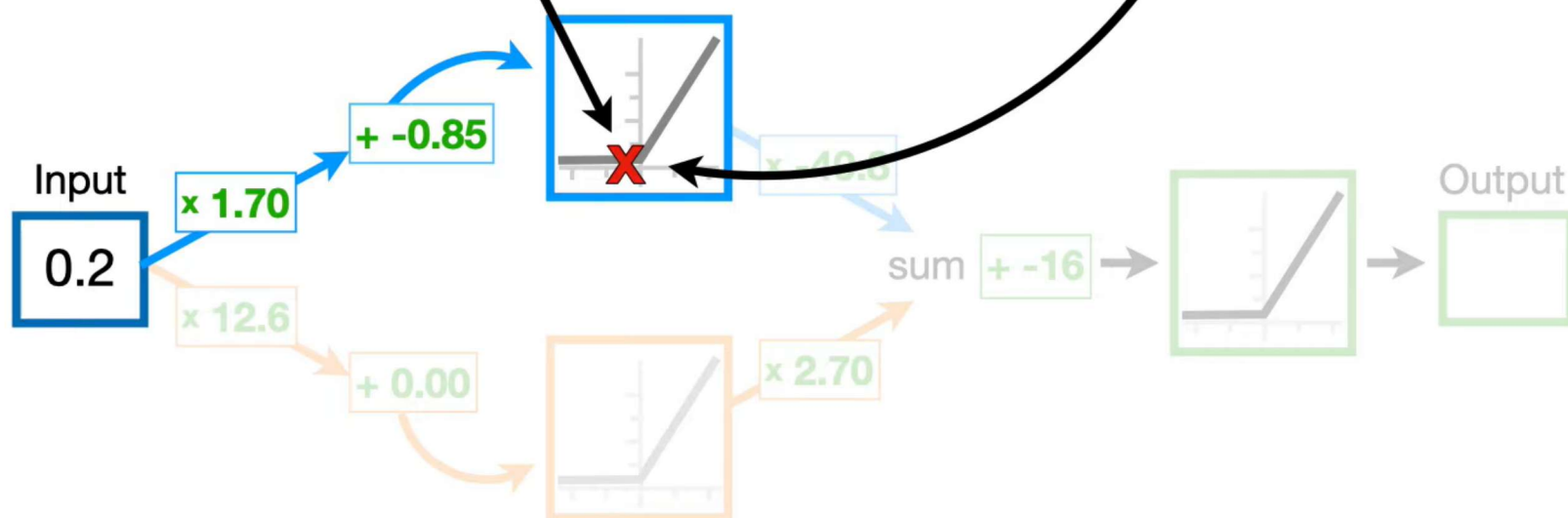
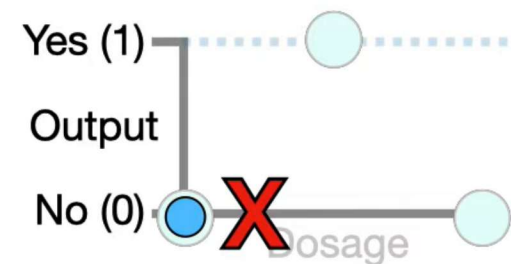




...and the corresponding
y-axis value is 0.

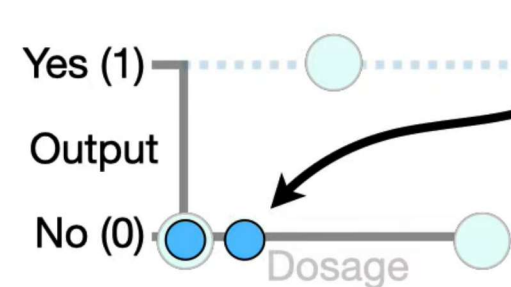
$$(0.1 \times 1.70) + -0.85 = -0.51$$

$$f(-0.51) = \max(0, -0.51) = 0$$



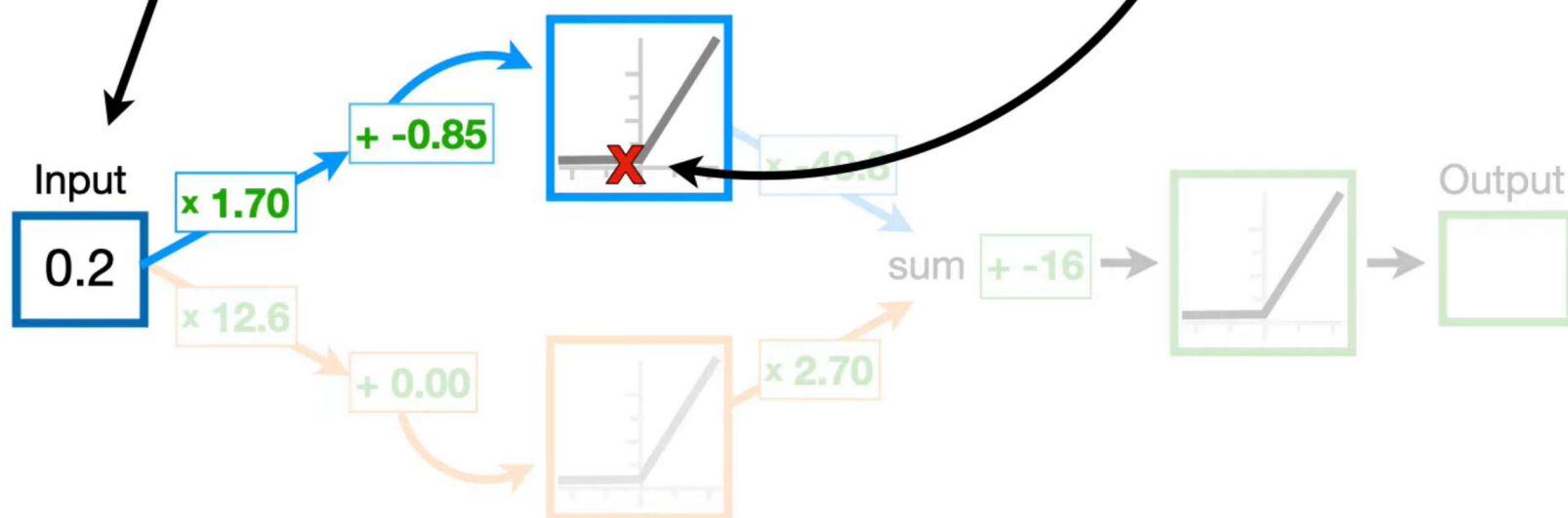


So let's put a **blue dot** at **0** for when **Dosage = 0.2**.



$$(0.1 \times 1.70) + -0.85 = -0.51$$

$$f(-0.51) = \max(0, -0.51) = 0$$

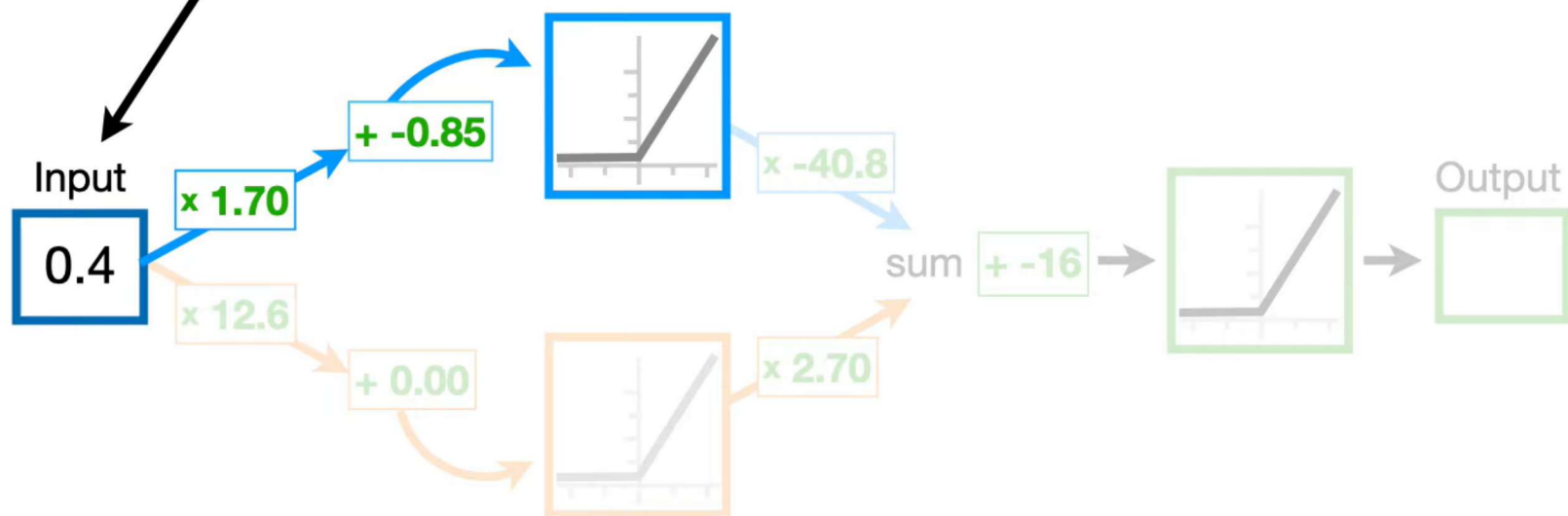
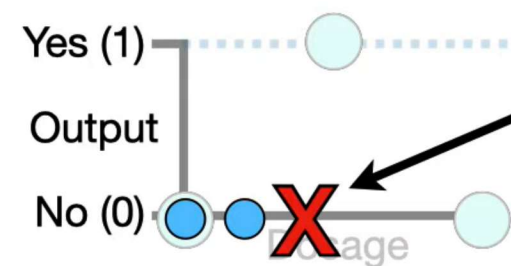




$$(\text{Dosage} \times 1.70) + -0.85 = \text{x-axis coordinate}$$

And if we increase the **Dosage** value to **0.4**...

$$f(x) = \max(0, x) = \text{y-axis coordinate}$$

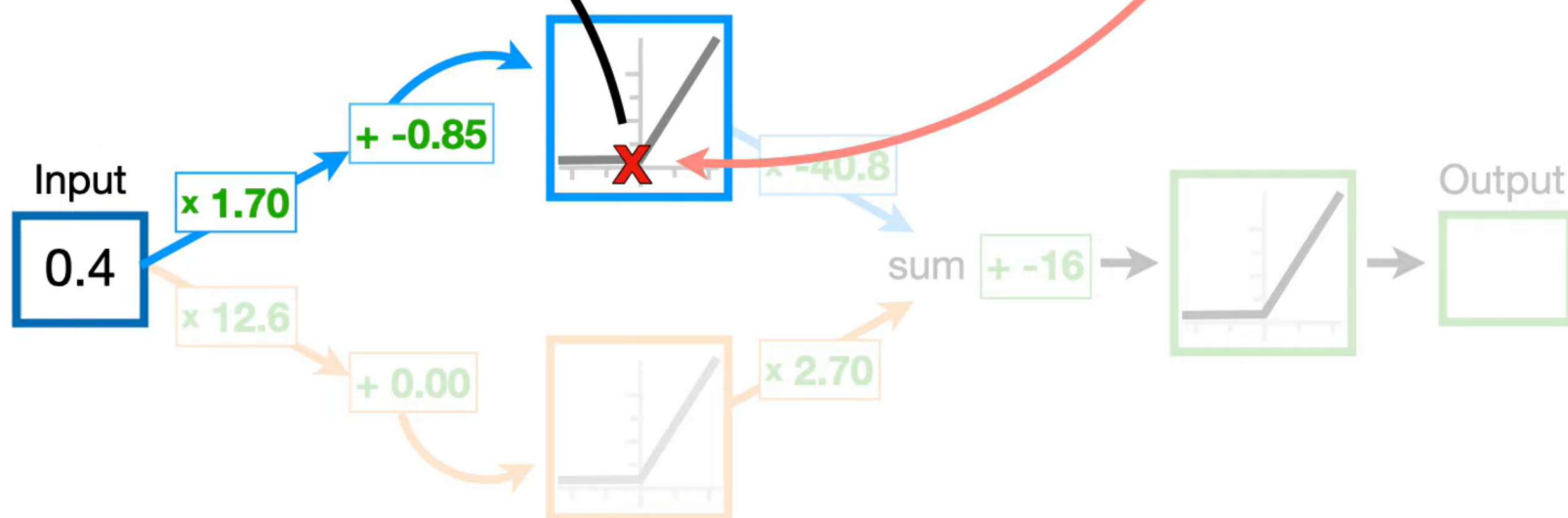
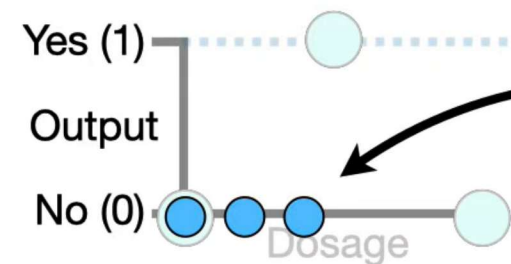




$$(\text{Dosage} \times 1.70) + -0.85 = \text{x-axis coordinate}$$

...we get **0** for the y-axis coordinate.

$$f(x) = \max(0, x) = \text{y-axis coordinate}$$

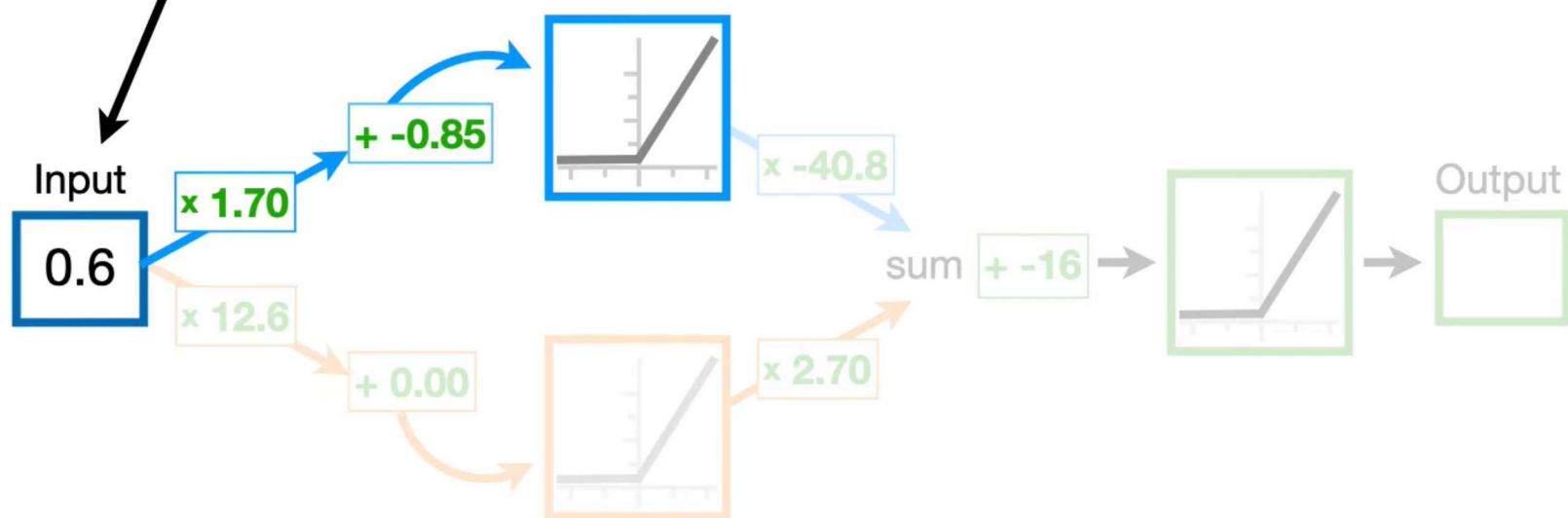
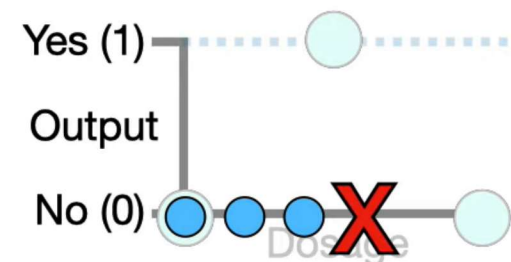




However, when **Dosage** = 0.6...

$$(0.6 \times 1.70) + -0.85 = \text{x-axis coordinate}$$

$$f(x) = \max(0, x) = \text{y-axis coordinate}$$

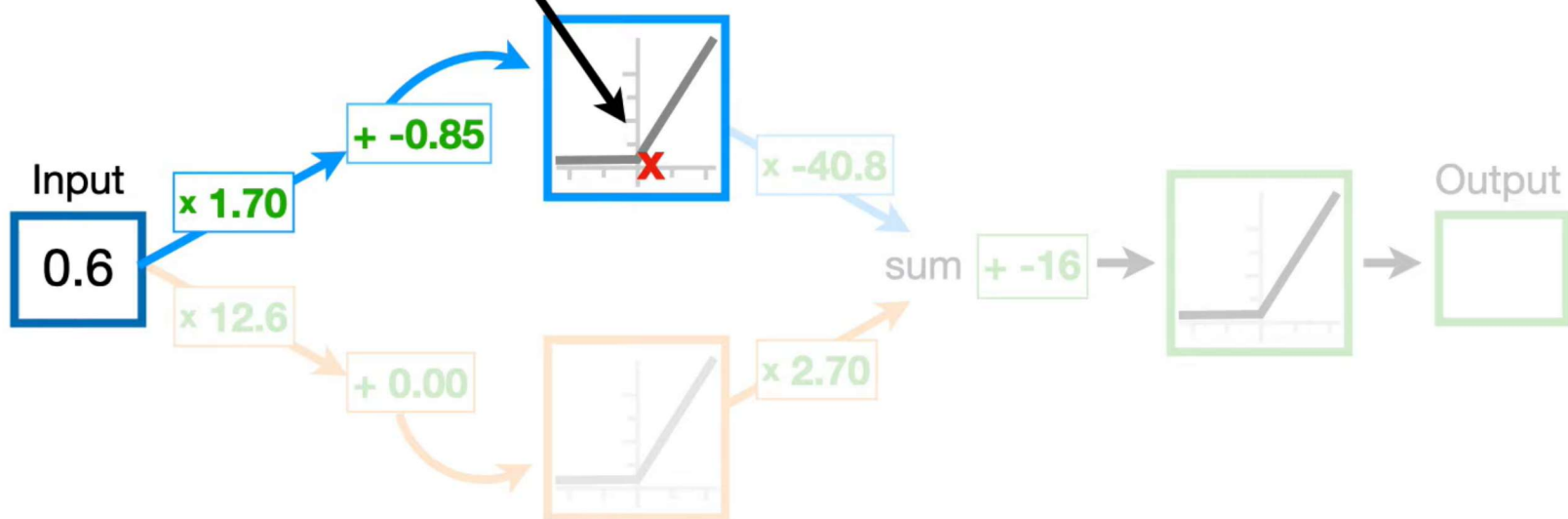
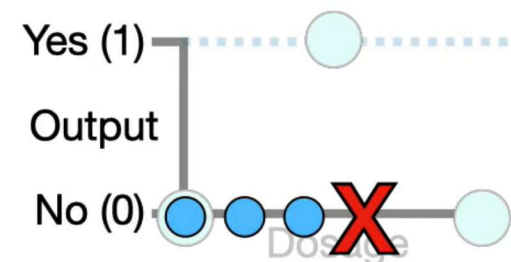




$$(0.6 \times 1.70) + -0.85 = 0.17$$

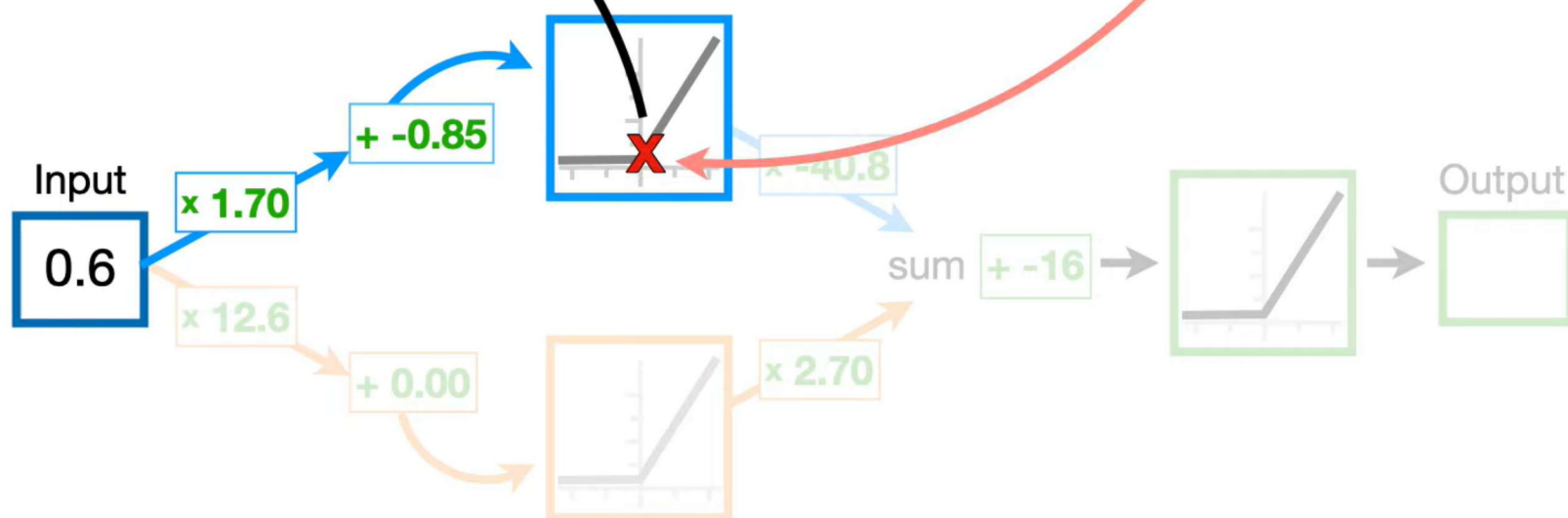
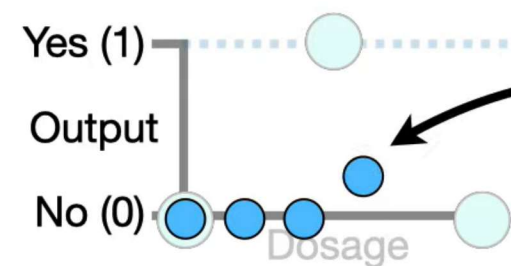
...the x-axis coordinate
is **0.17**.

$f(x) = \max(0, x) =$ y-axis coordinate





...and the corresponding
y-axis value is **0.17**.

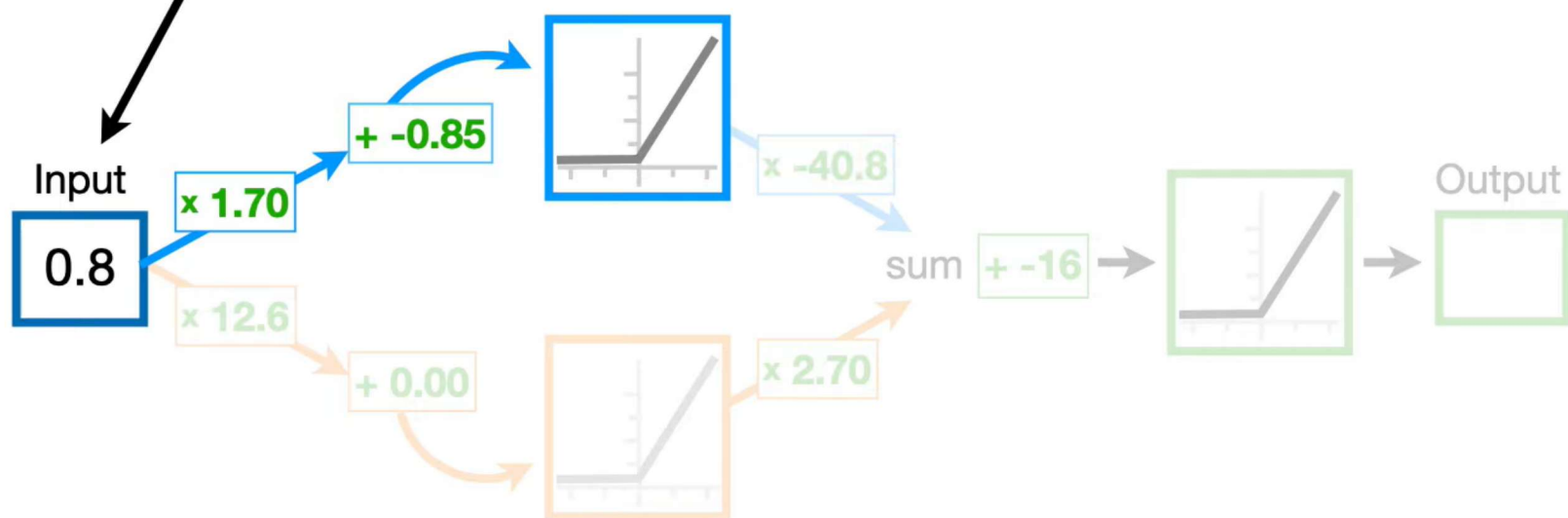
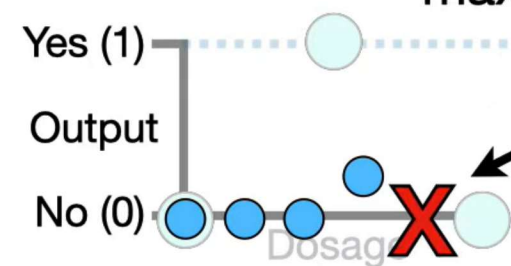




And if we continue to increase the **Dosage** values all the way to 1 (the maximum **Dosage**)...

$$(\text{Dosage} \times 1.70) + -0.85 = \text{x-axis coordinate}$$

$$f(x) = \max(0, x) = \text{y-axis coordinate}$$

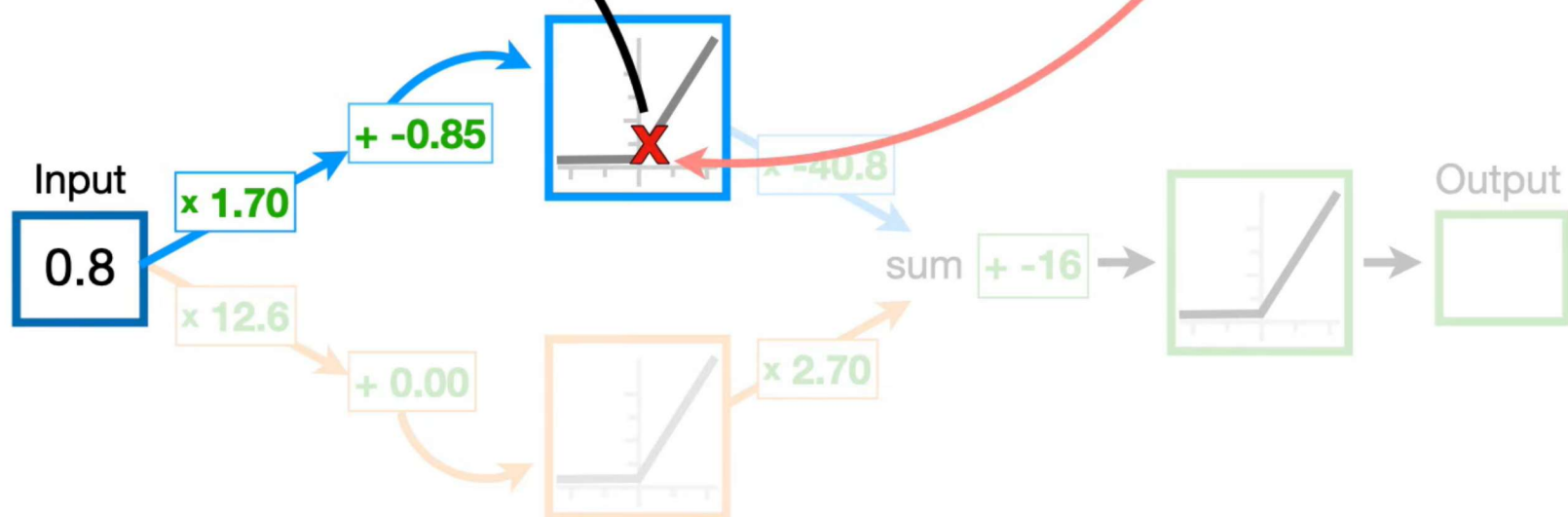
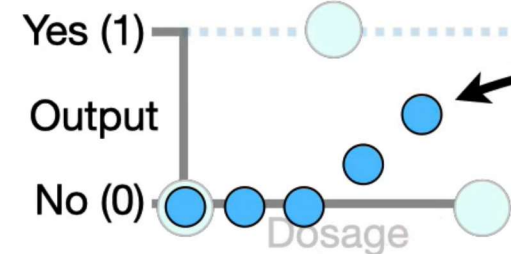




And if we continue to increase the **Dosage** values all the way to 1 (the maximum **Dosage**)...

$$(\text{Dosage} \times 1.70) + -0.85 = \text{x-axis coordinate}$$

$$f(x) = \max(0, x) = \text{y-axis coordinate}$$

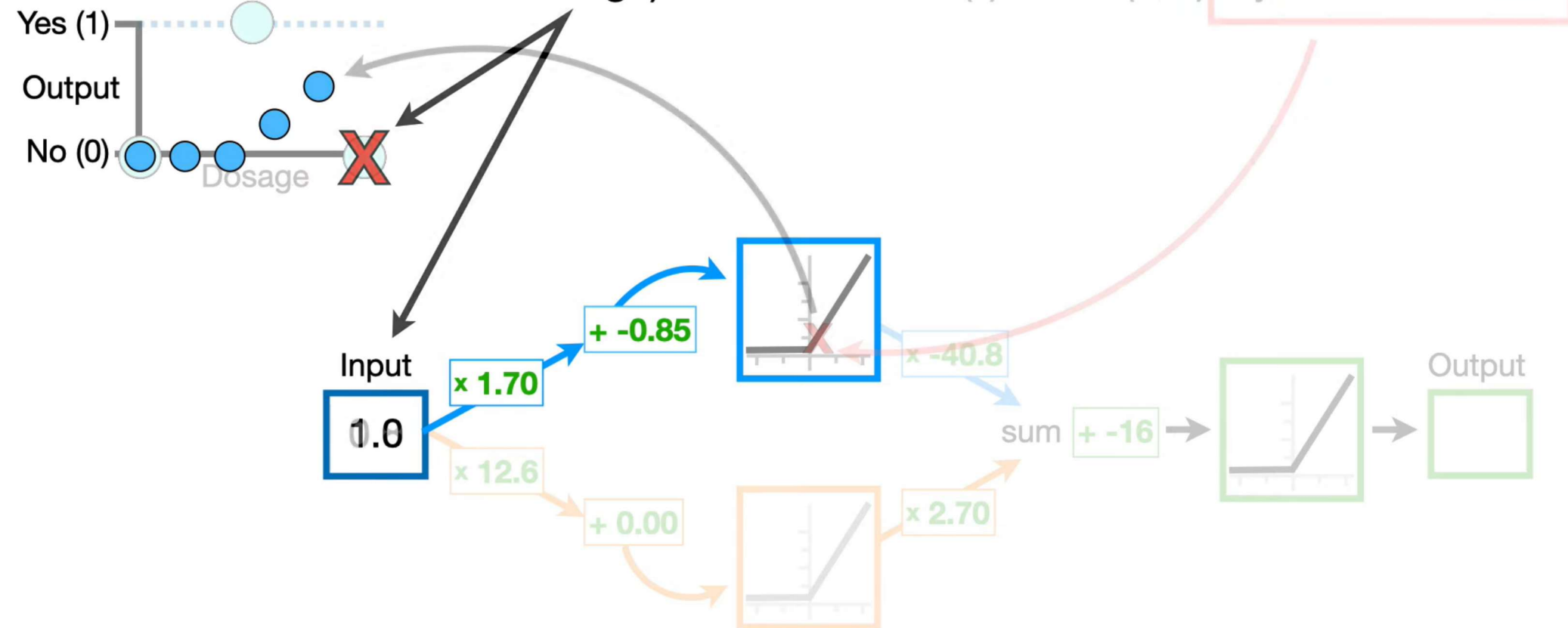




And if we continue to increase the **Dosage** values all the way to 1 (the maximum **Dosage**)...

$$(\text{Dosage} \times 1.70) + -0.85 = \text{x-axis coordinate}$$

$$f(x) = \max(0, x) = \text{y-axis coordinate}$$

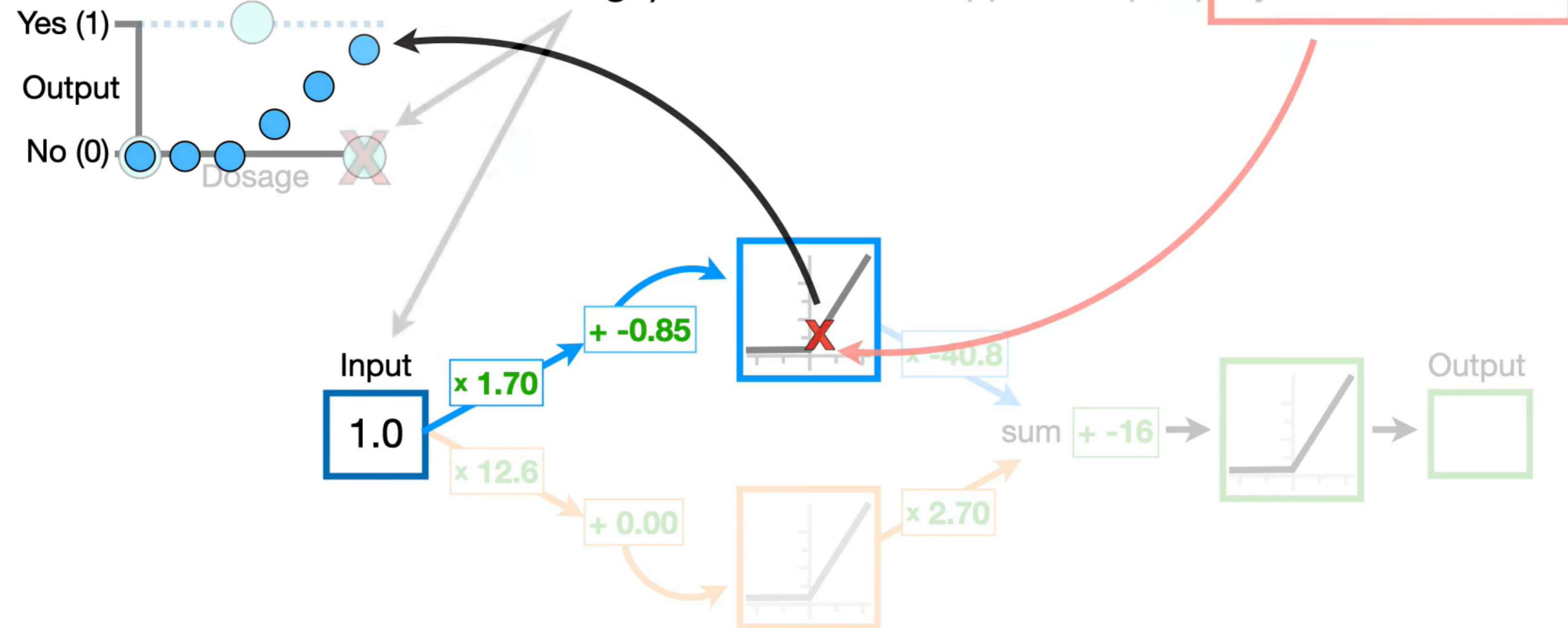




And if we continue to increase the **Dosage** values all the way to 1 (the maximum **Dosage**)...

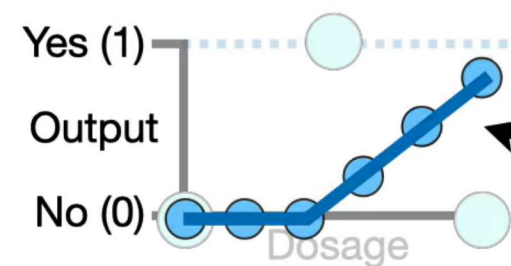
$$(\text{Dosage} \times 1.70) + -0.85 = \text{x-axis coordinate}$$

$$f(x) = \max(0, x) = \text{y-axis coordinate}$$



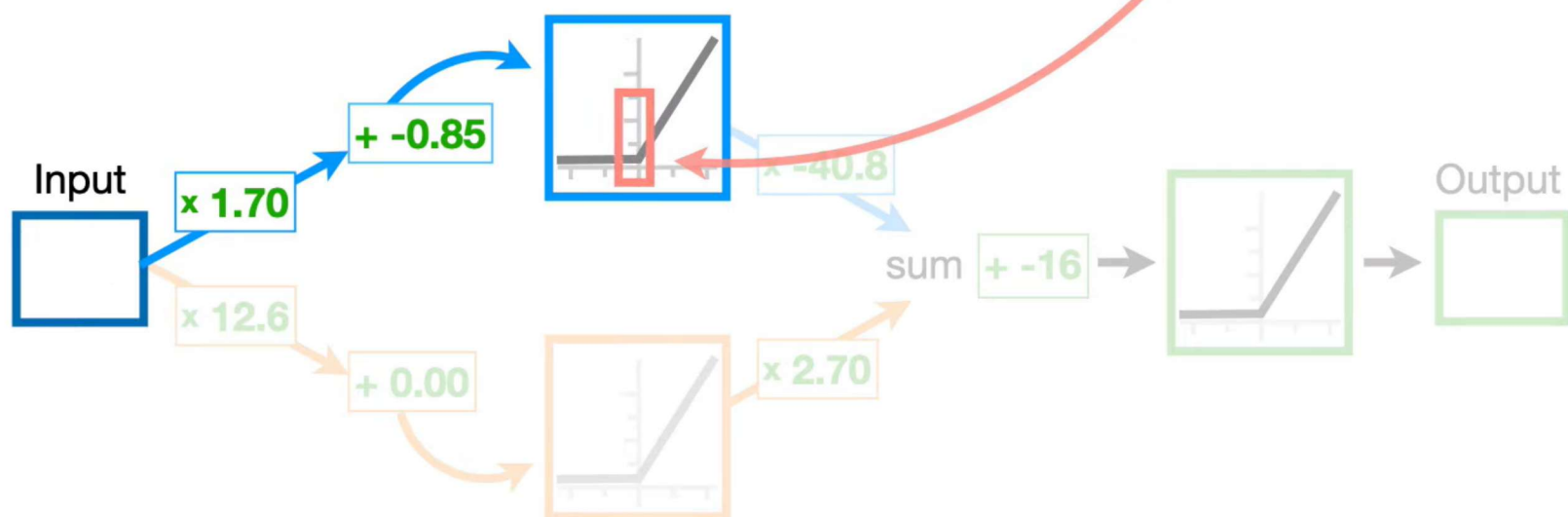


...we get this
bent blue line.



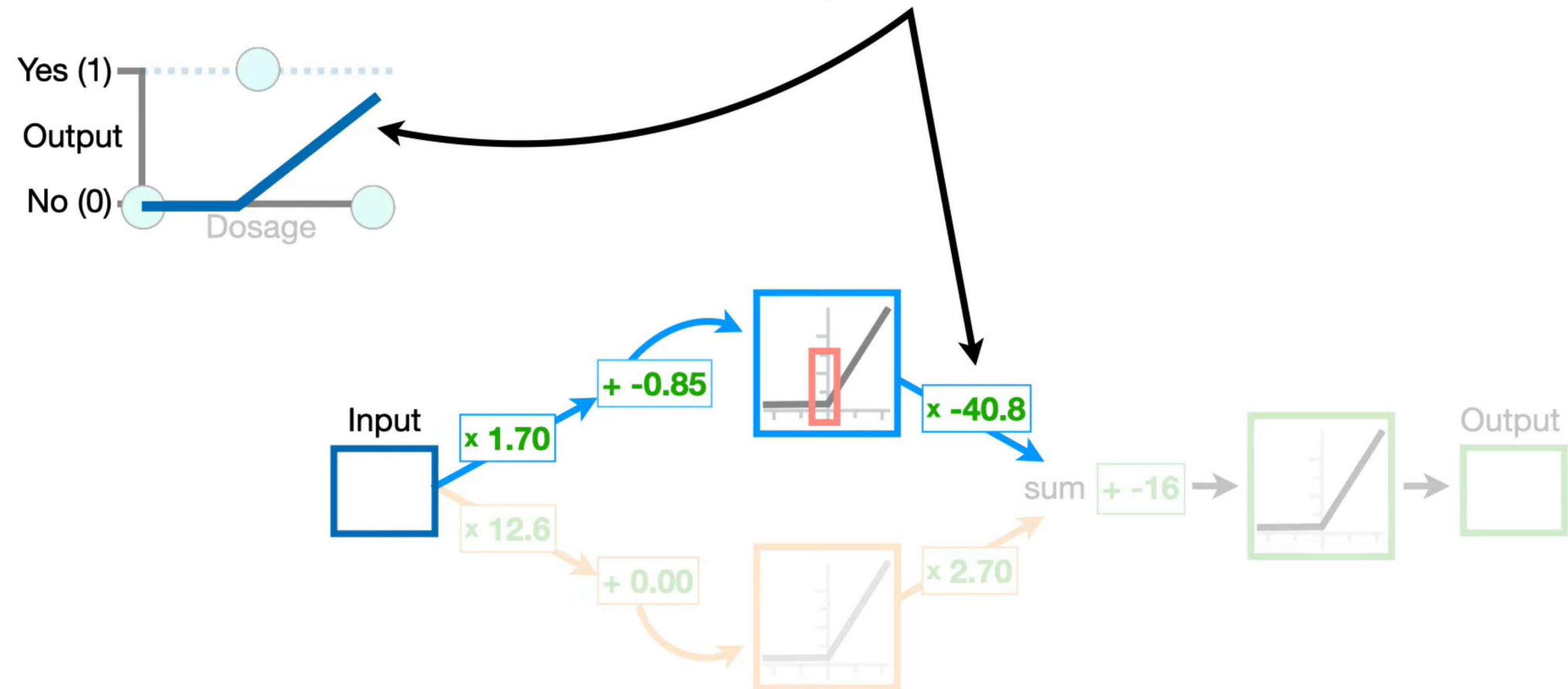
$$(\text{Dosage} \times 1.70) + -0.85 = \text{x-axis coordinate}$$

$$f(x) = \max(0, x) = \text{y-axis coordinate}$$



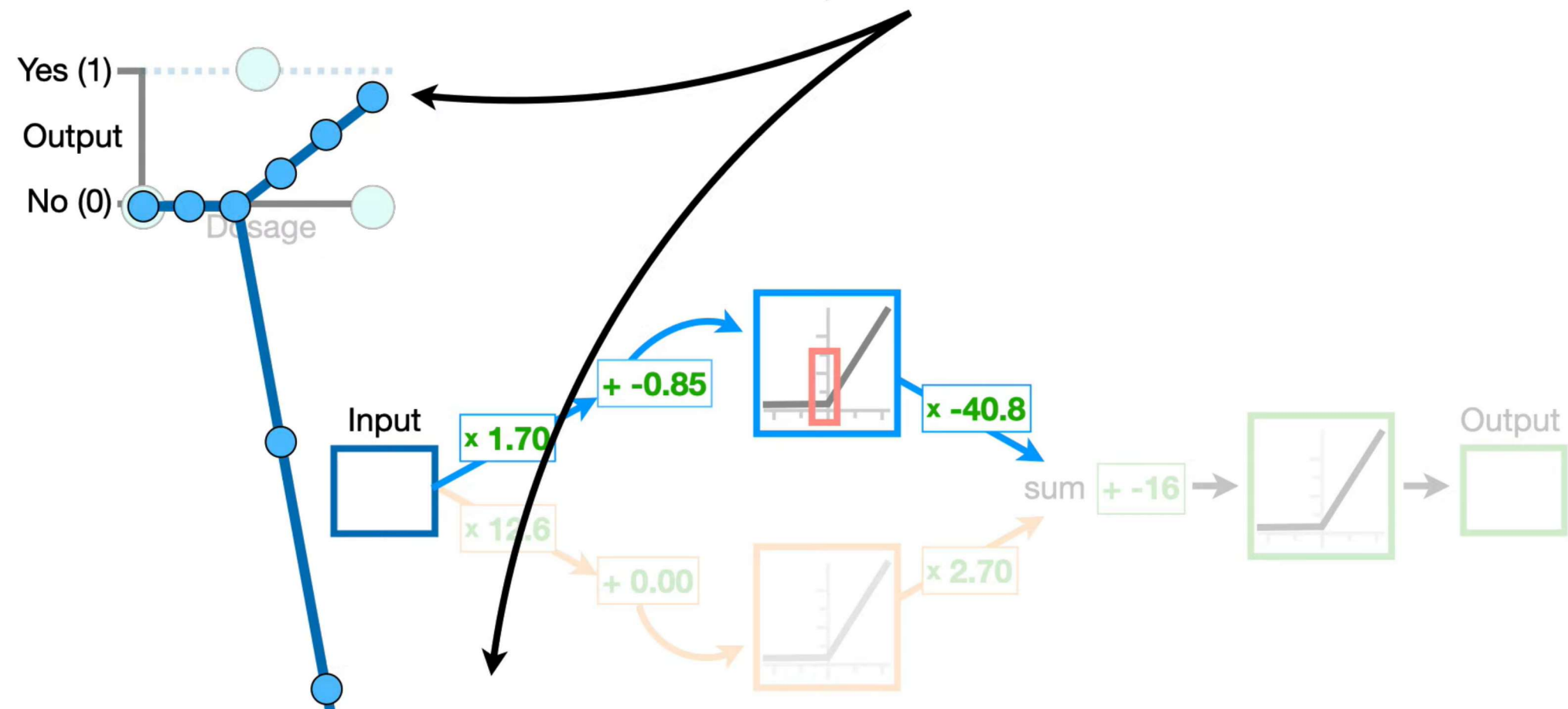


Then we multiply the y-axis coordinates on the **bent blue line** by **-40.8**...





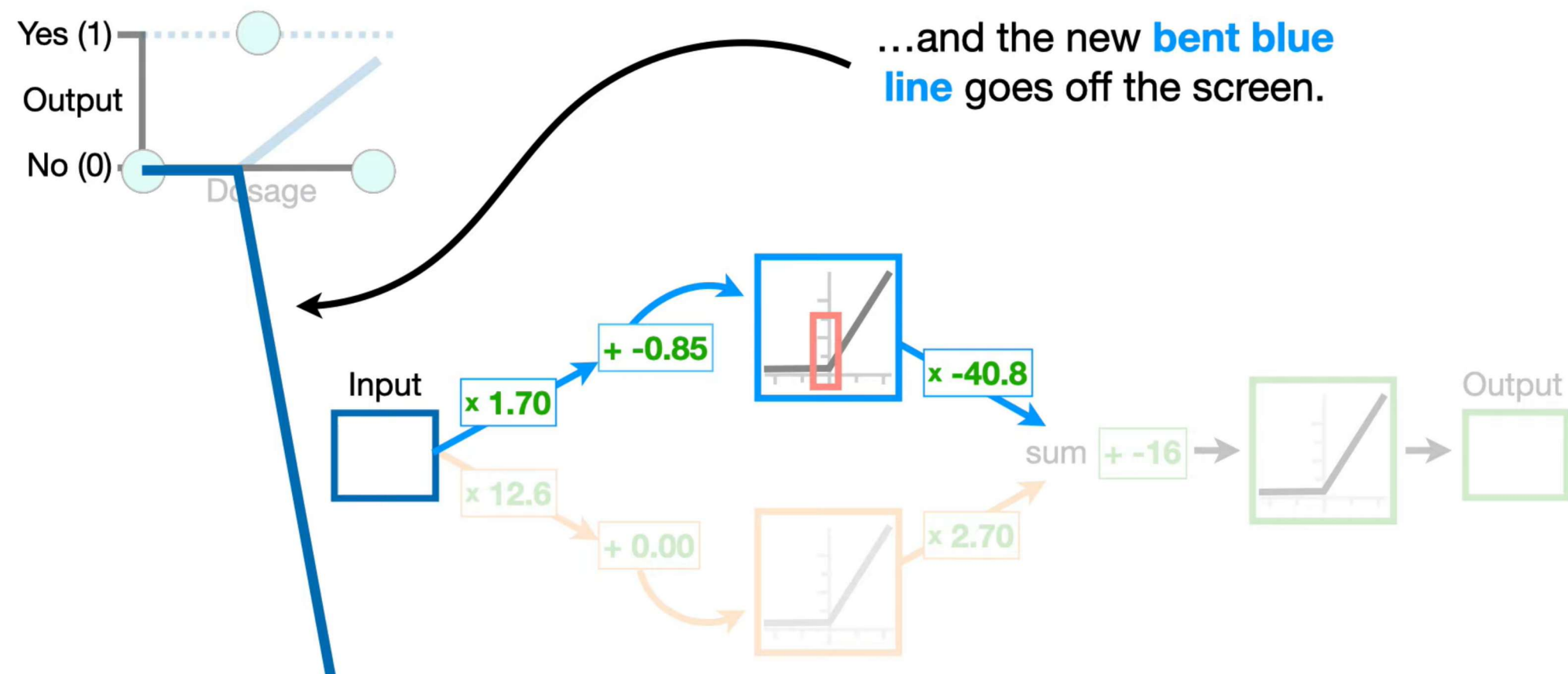
Then we multiply the y-axis coordinates on the **bent blue line** by **-40.8...**





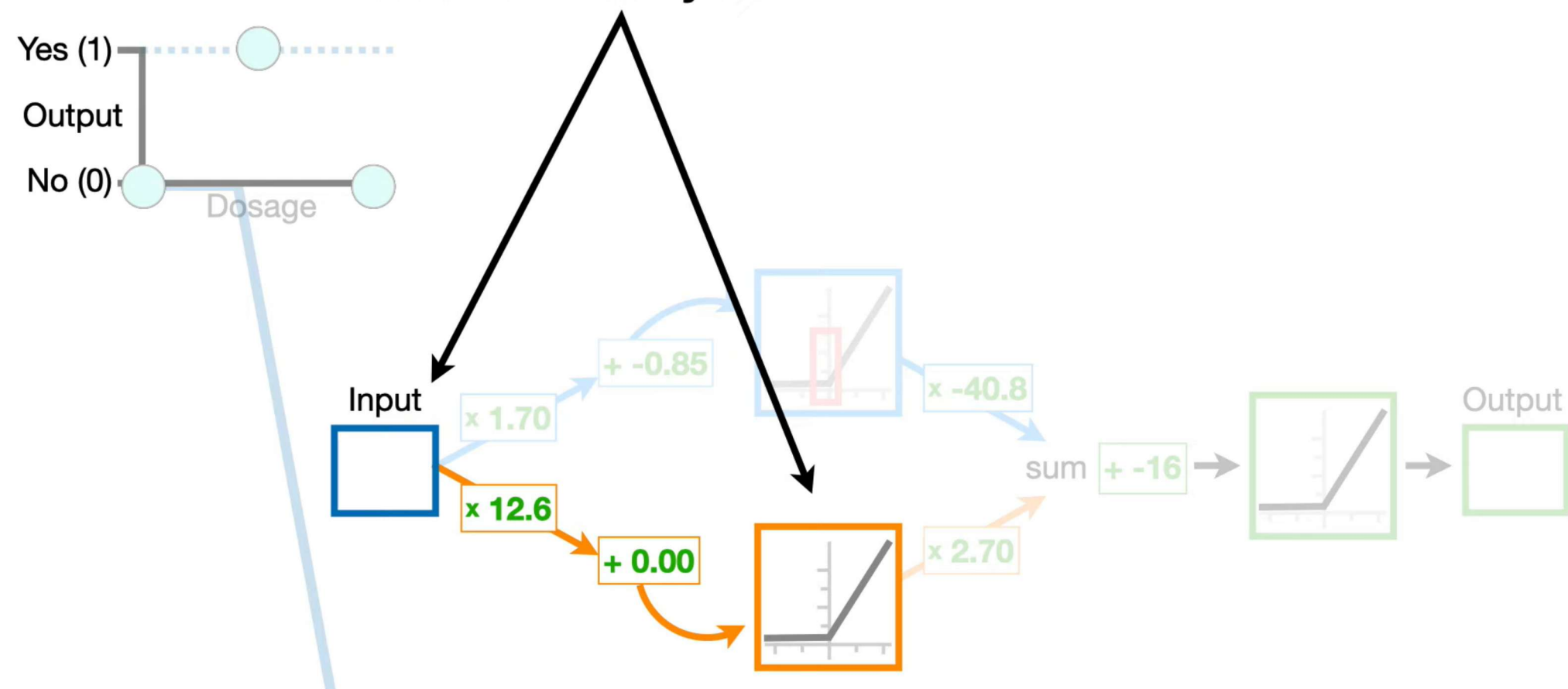
Then we multiply the y-axis coordinates on the **bent blue line** by **-40.8**...

...and the new **bent blue line** goes off the screen.





Now when we run **Dosages** through the connection to the bottom **Node** in the **Hidden Layer**...

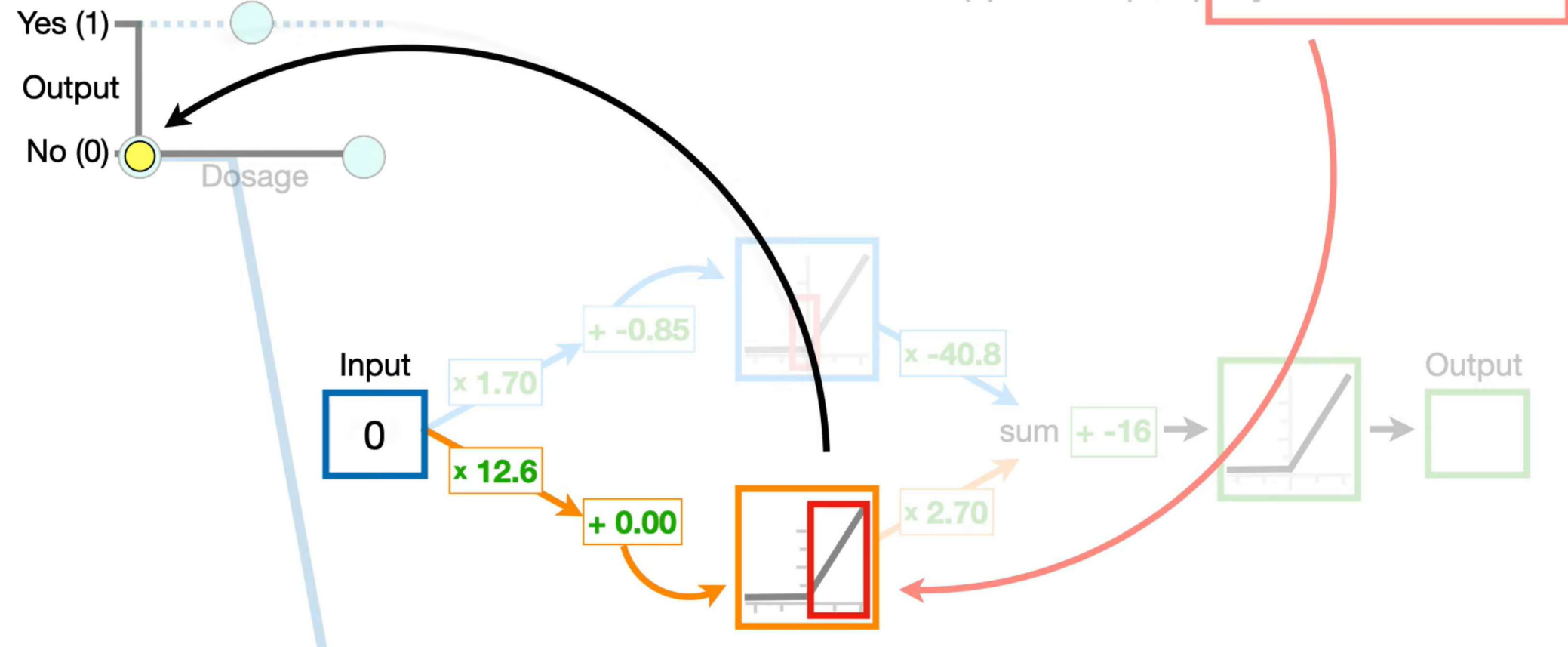




...we get the corresponding
y-axis coordinates...

$$(\text{Dosage} \times 12.6) + 0.00 = \text{x-axis coordinate}$$

$$f(x) = \max(0, x) = \text{y-axis coordinate}$$

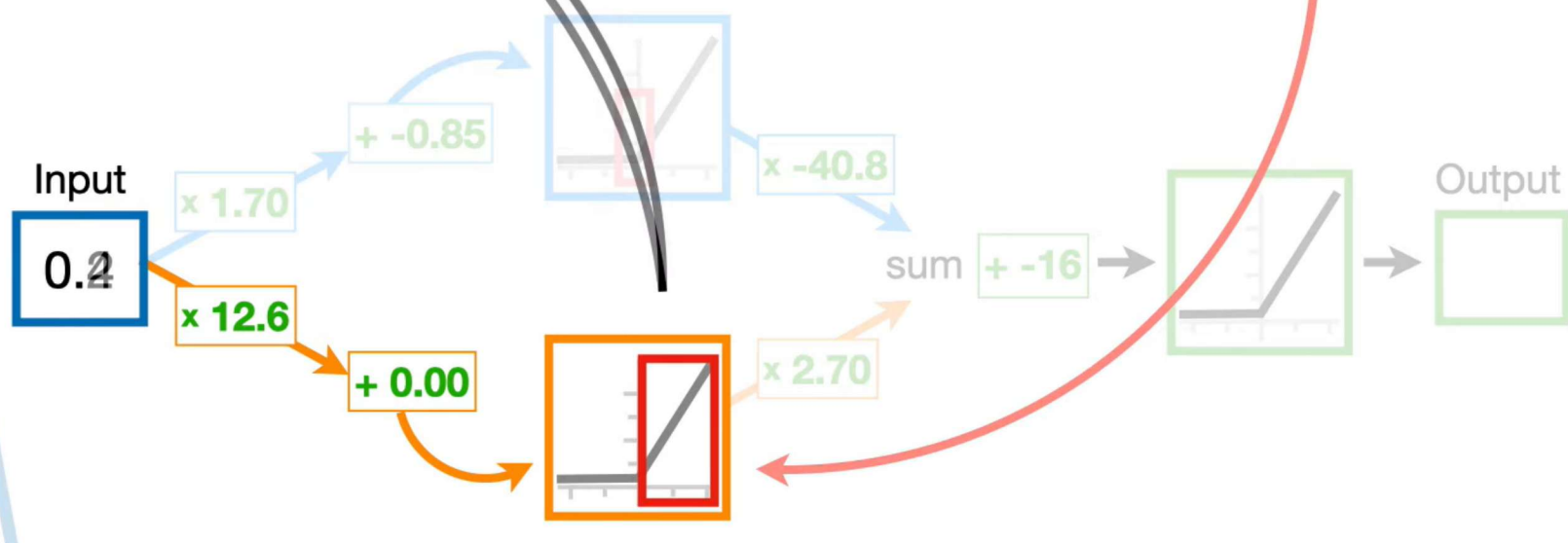
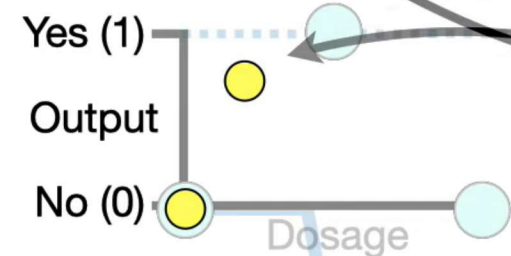


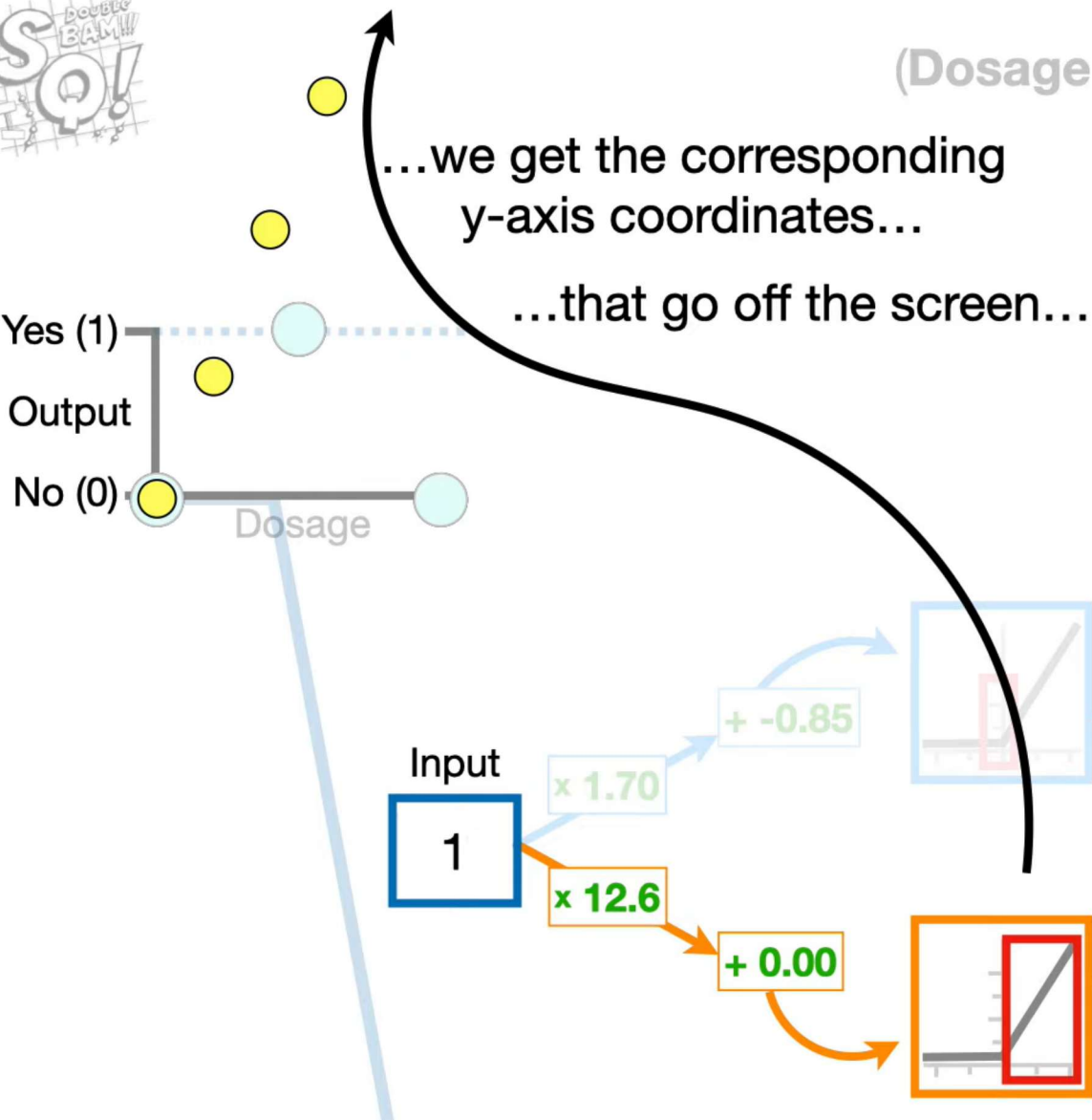


$$(\text{Dosage} \times 12.6) + 0.00 = \text{x-axis coordinate}$$

...we get the corresponding
y-axis coordinates...

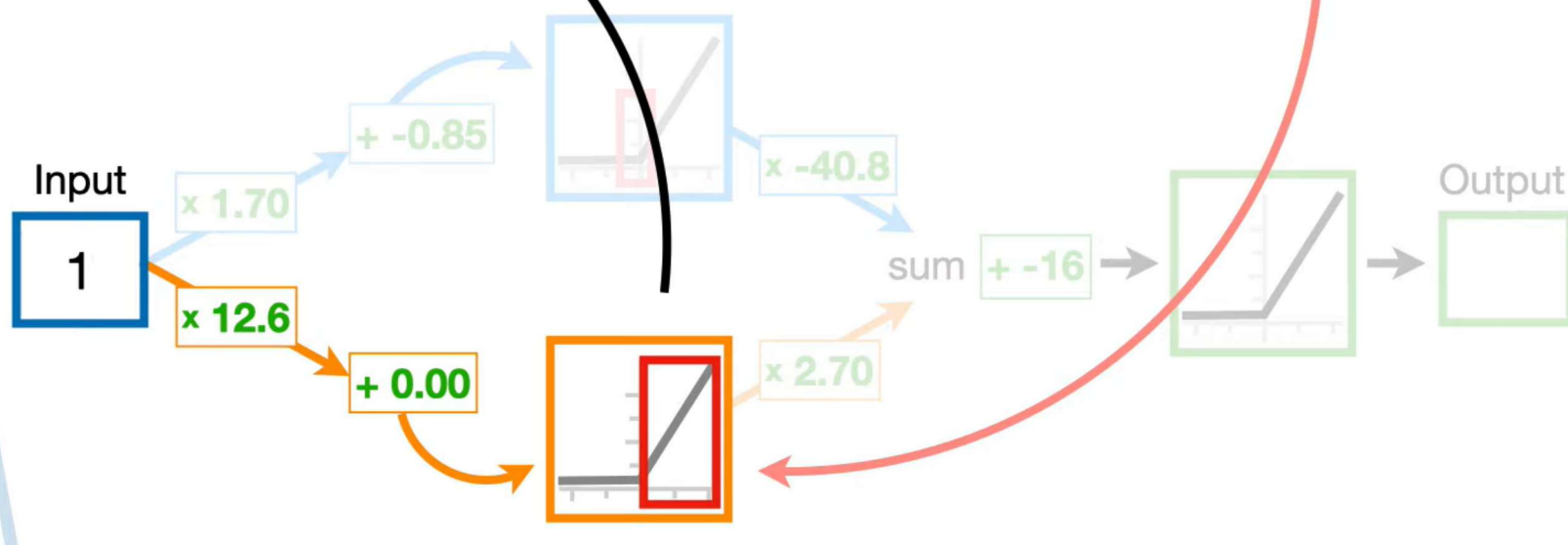
$$f(x) = \max(0, x) = \text{y-axis coordinate}$$





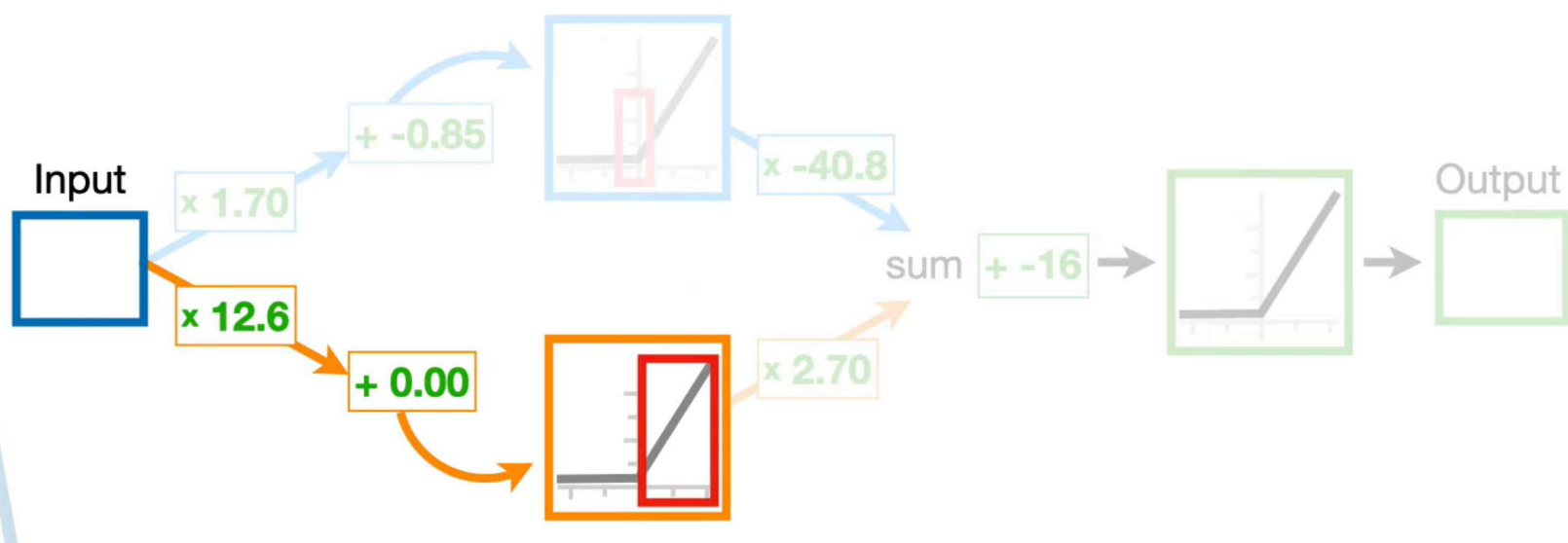
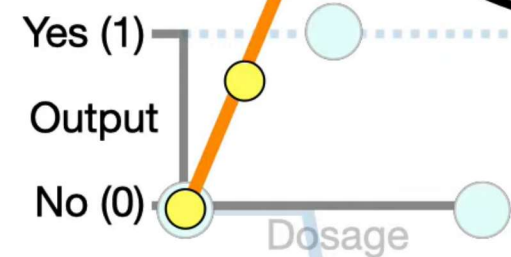
$$(\text{Dosage} \times 12.6) + 0.00 = \text{x-axis coordinate}$$

$$f(x) = \max(0, x) = \text{y-axis coordinate}$$



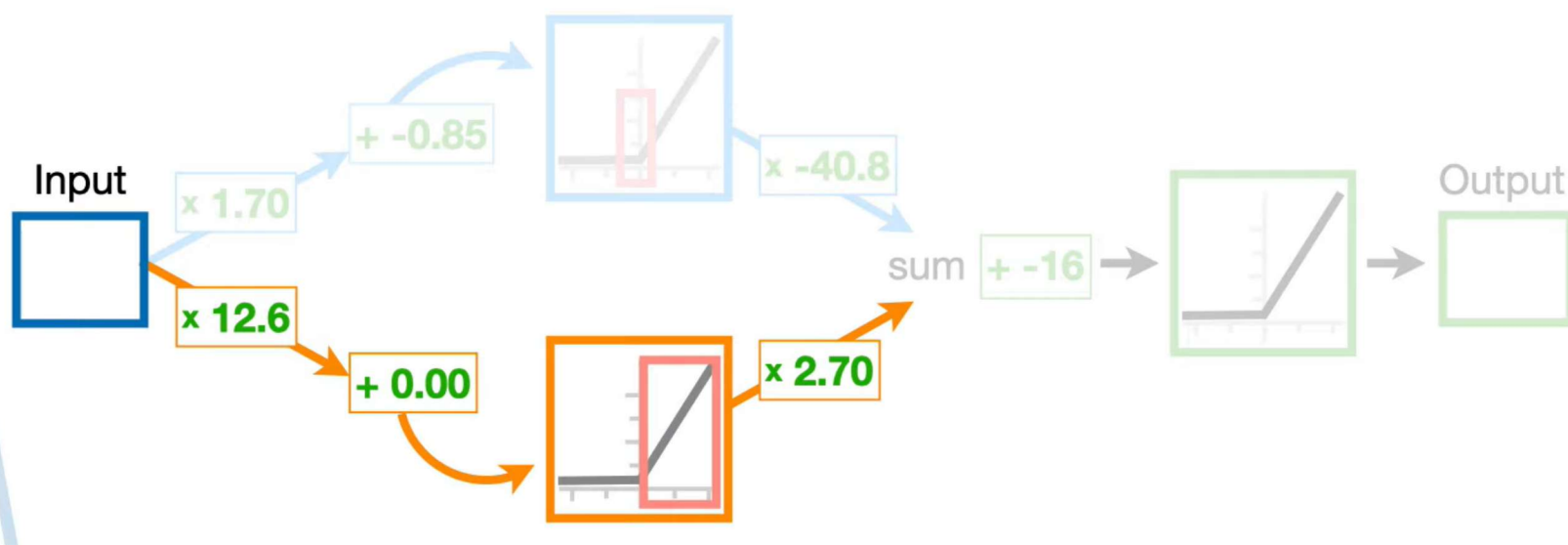
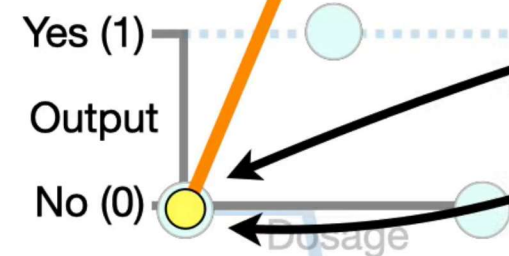


...for this
straight orange line.



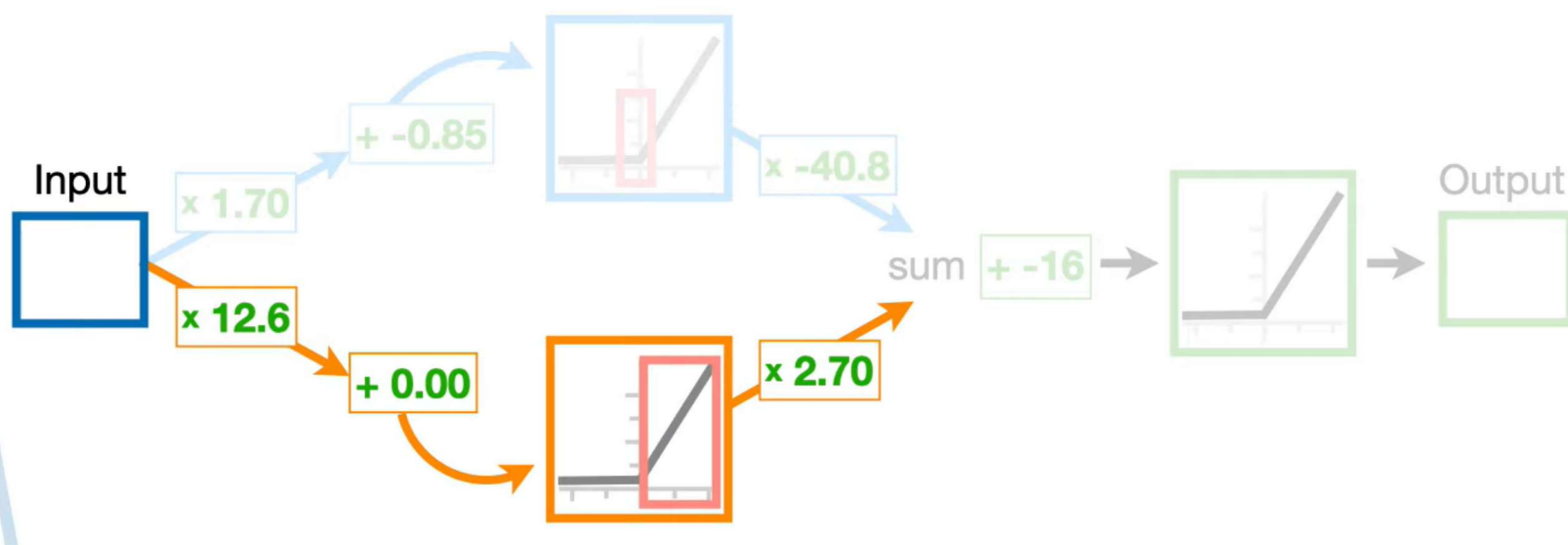
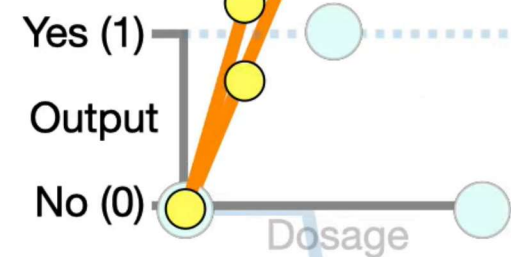


Now we multiply the y-axis coordinates on the **straight orange line** by **2.70**...



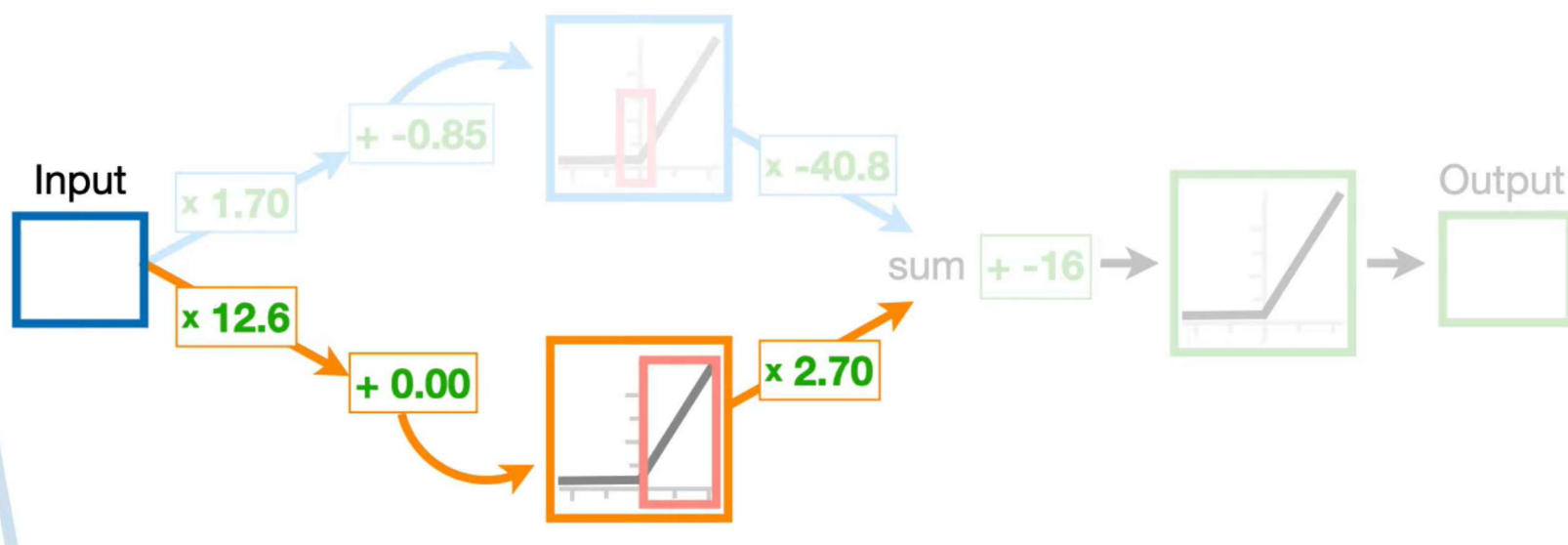
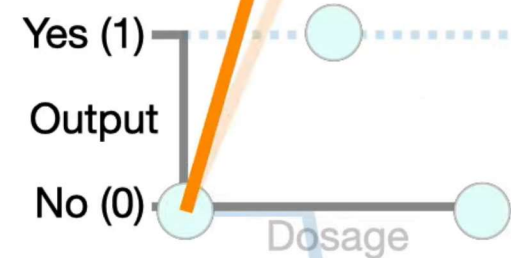


Now we multiply the y-axis coordinates on the **straight orange line** by **2.70...**



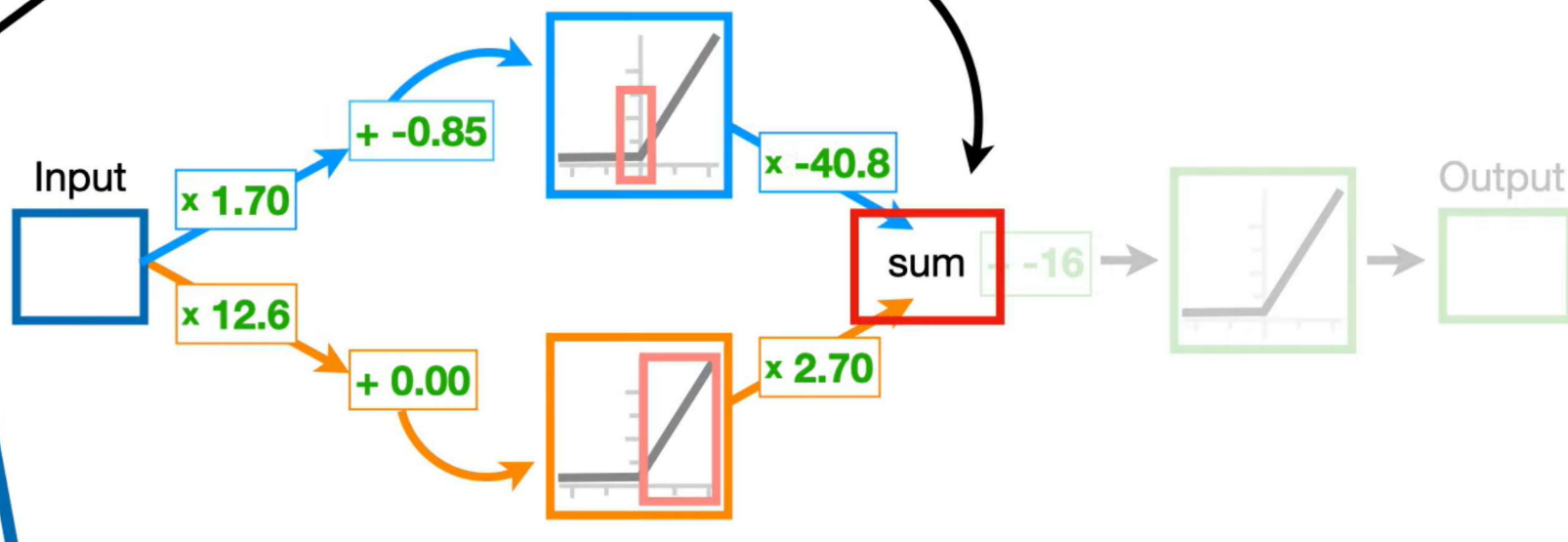
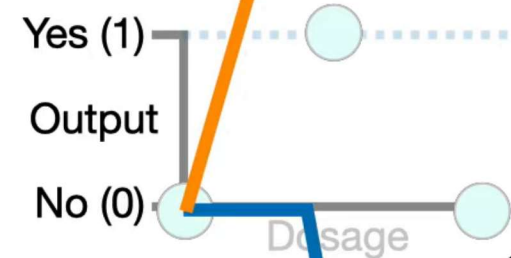


...and we end up with this final **straight orange line**.



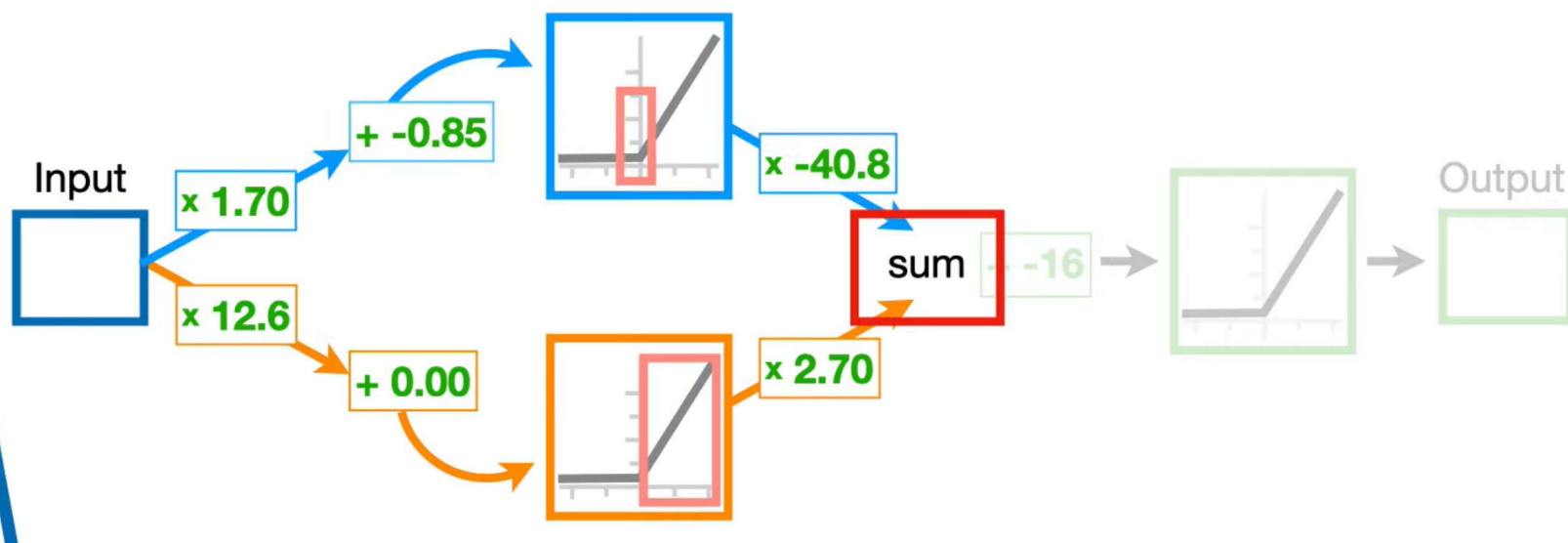
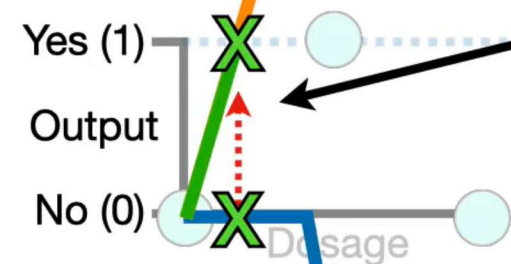


Now we add the **bent blue line** and **straight orange line** together...





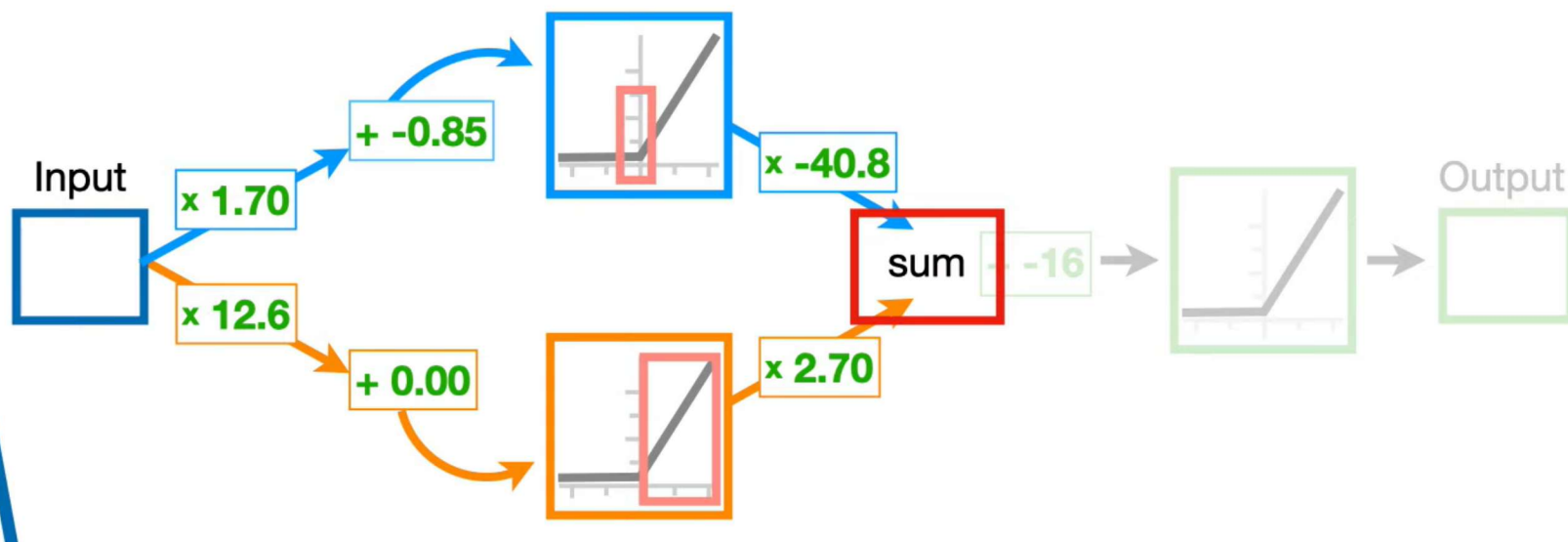
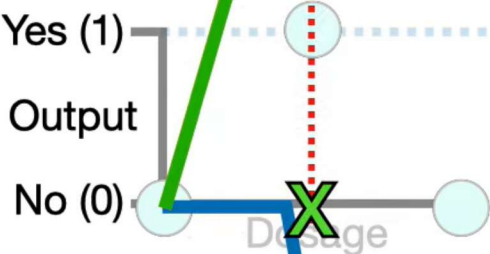
Now we add the **bent blue line** and **straight orange line** together...





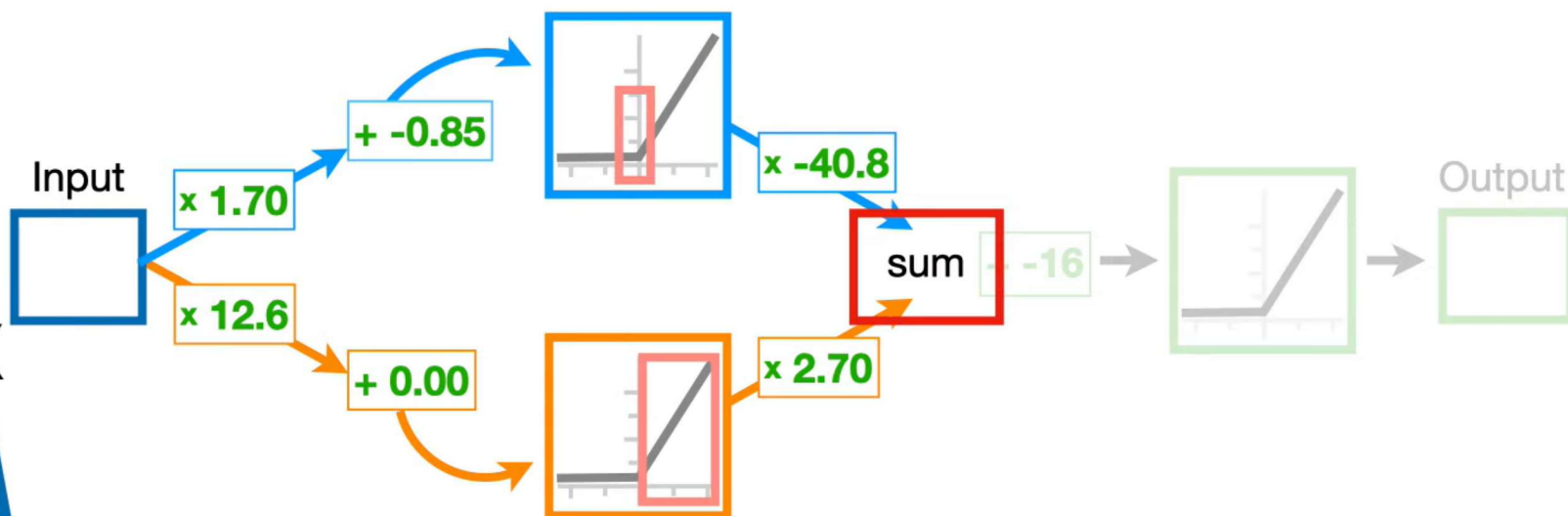
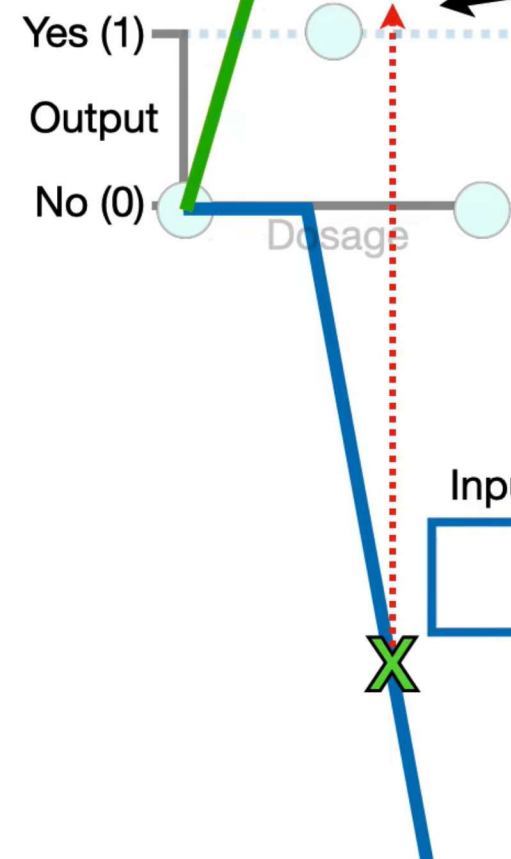
X

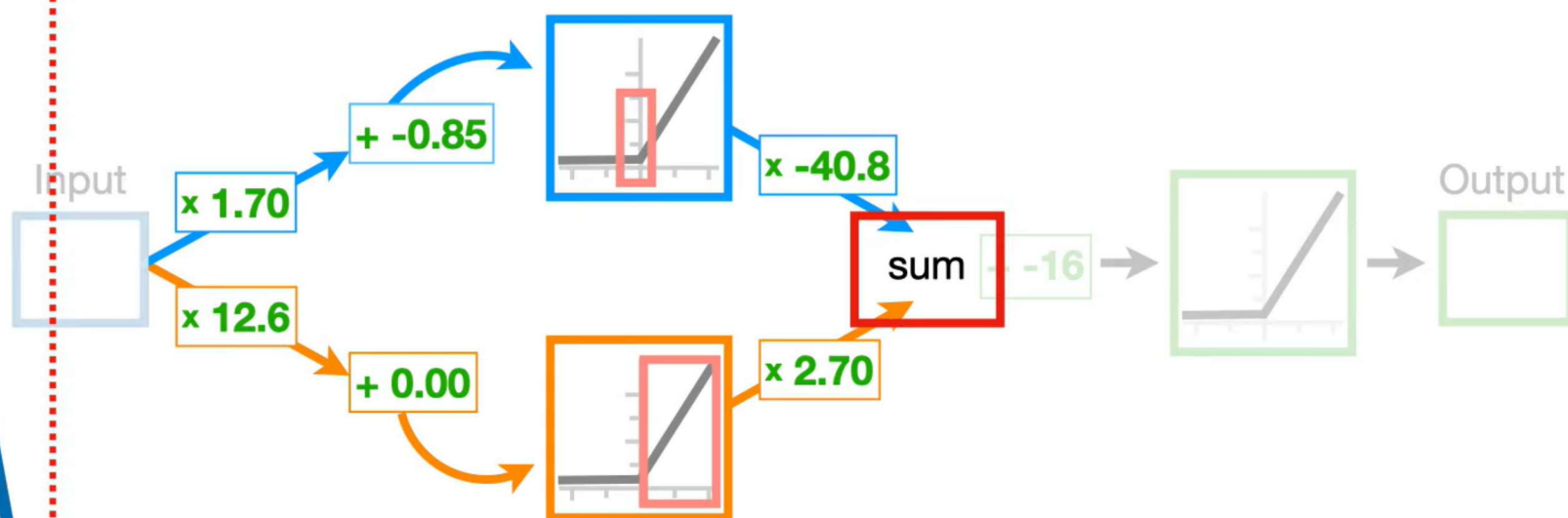
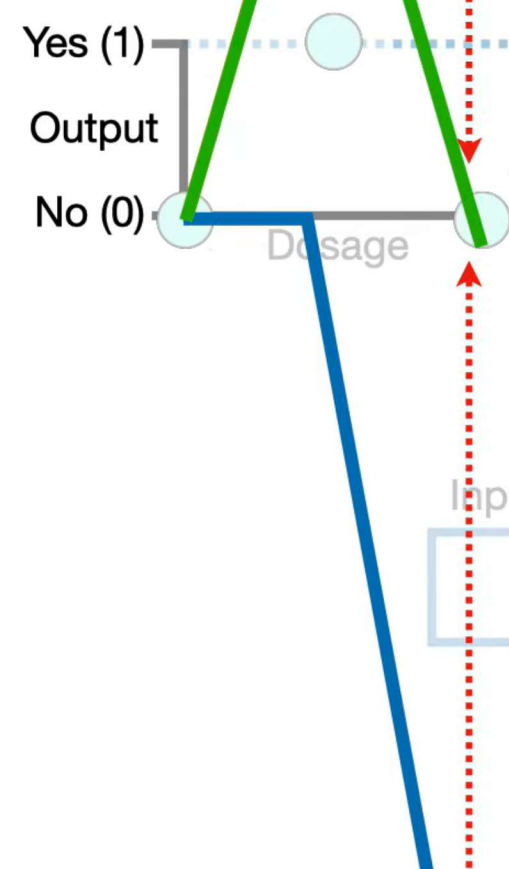
Now we add the **bent blue line** and **straight orange line** together...





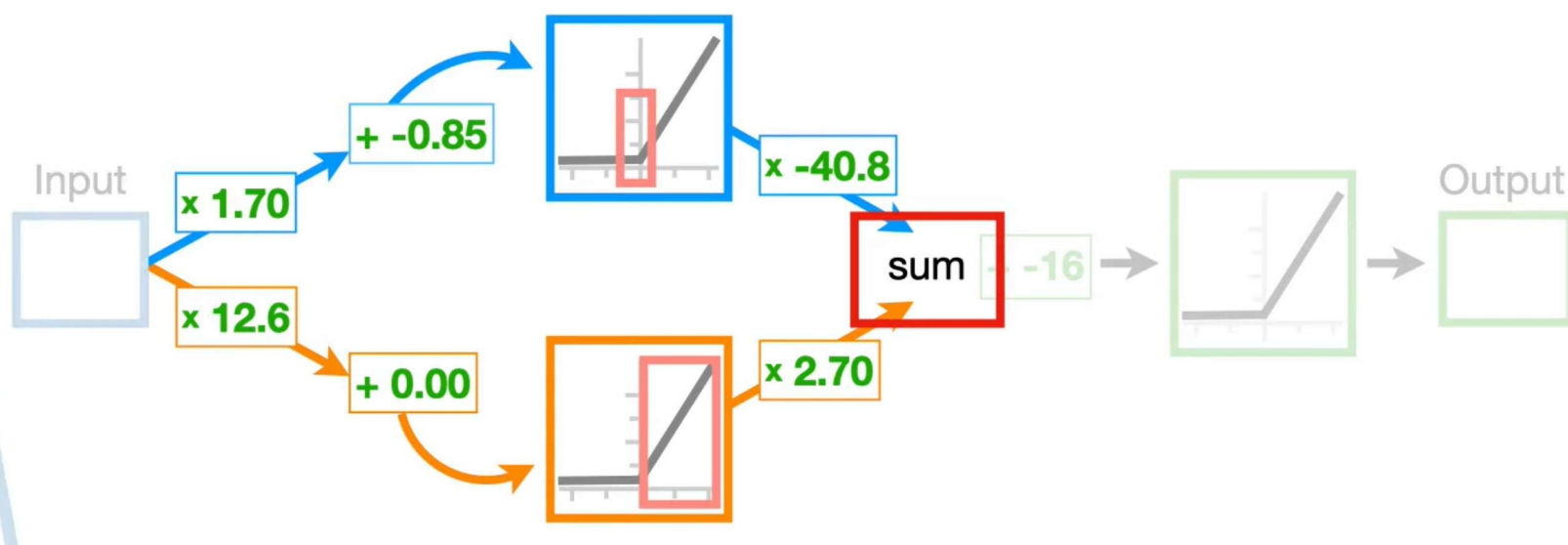
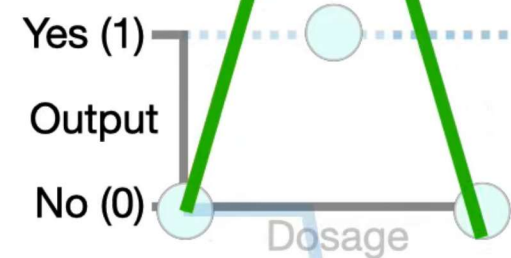
Now we add the **bent blue line** and **straight orange line** together...





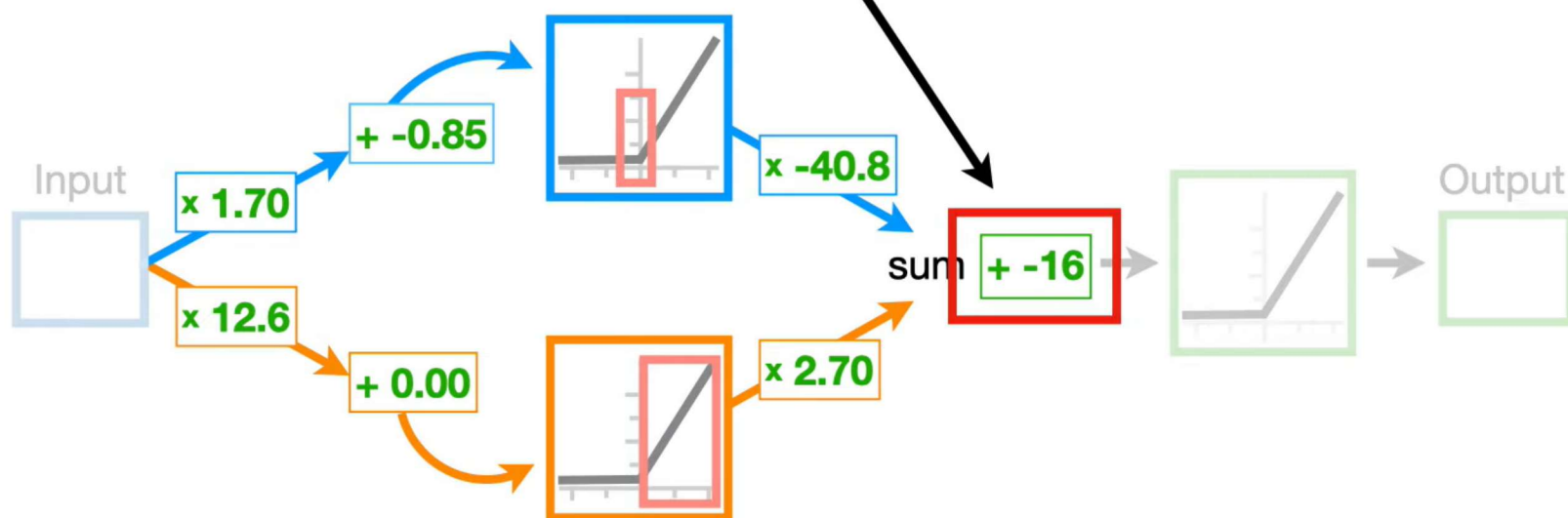
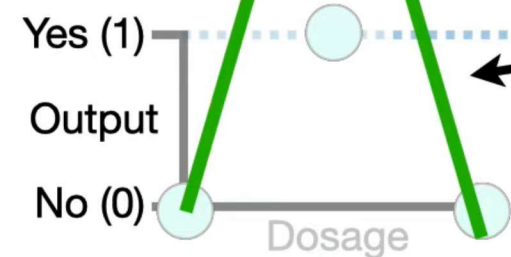


...to get this **green wedge**.



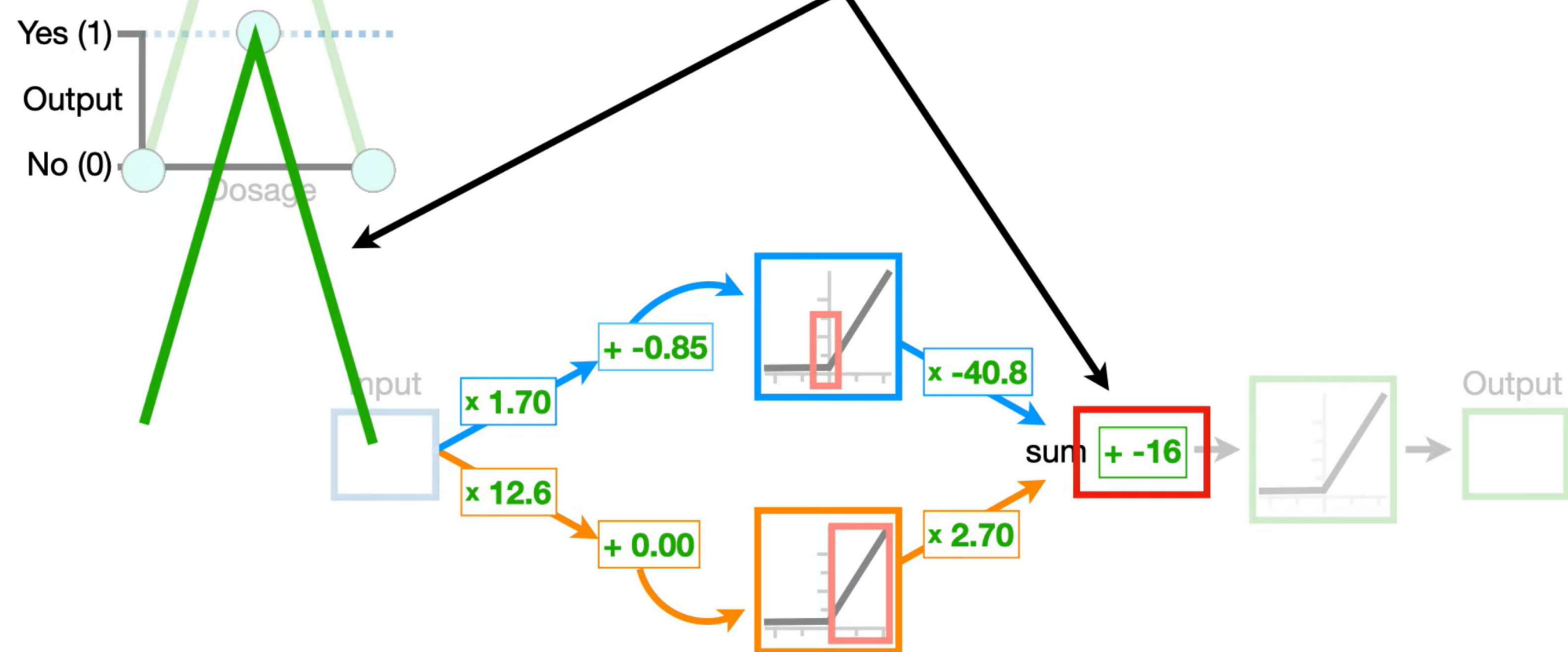


Now we add the final bias term, **-16**, to the y-axis coordinates on the **green wedge**.



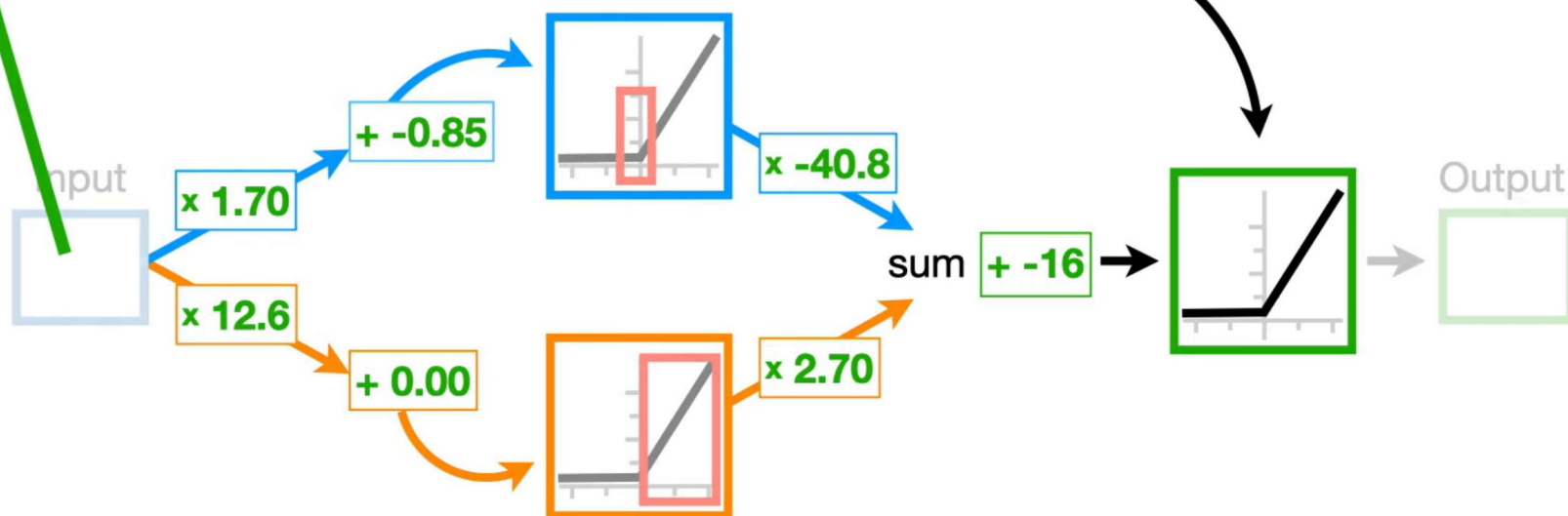
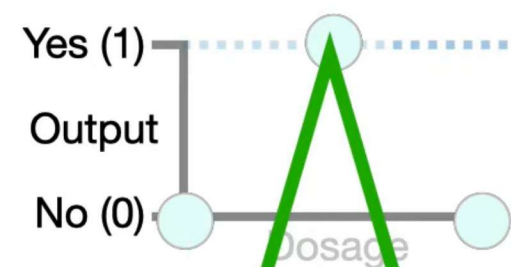


Now we add the final bias term, **-16**, to the y-axis coordinates on the **green wedge**.





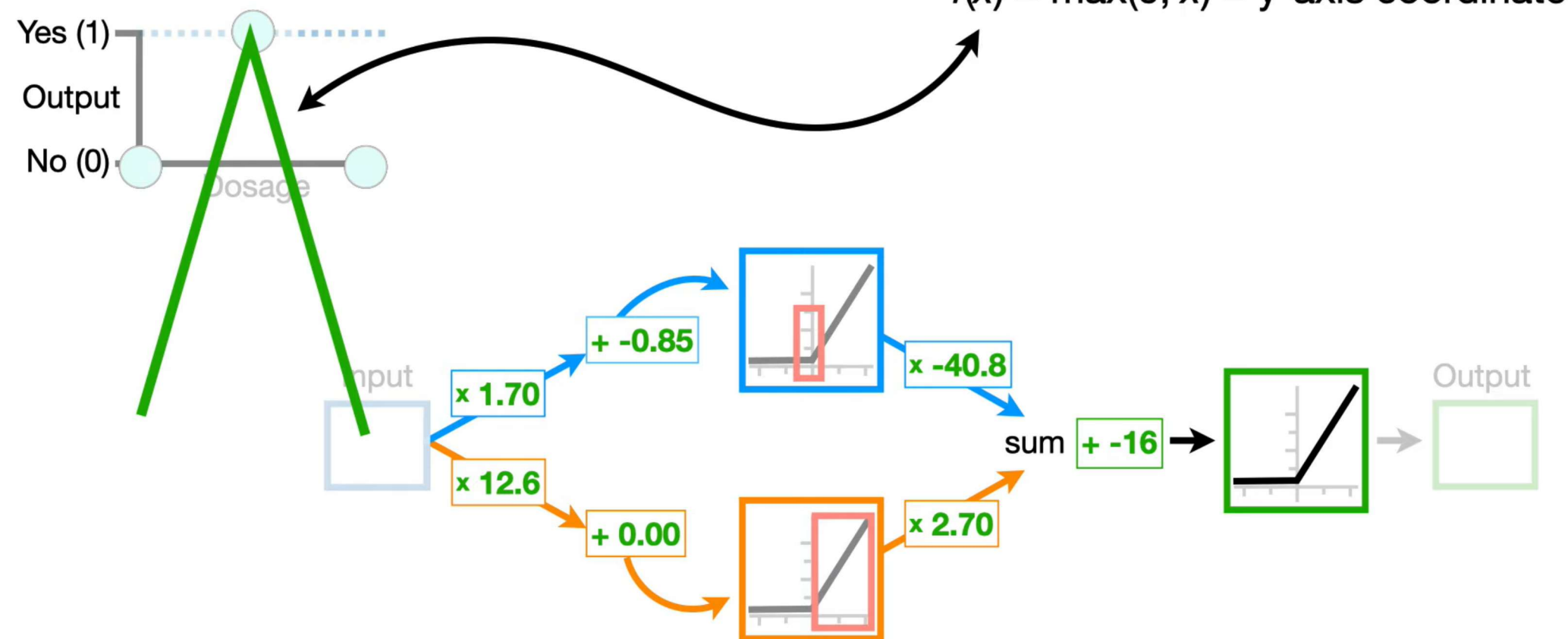
Lastly, because we included the **ReLU** activation function right in front of the output...





...we use the **green wedge** as its input.

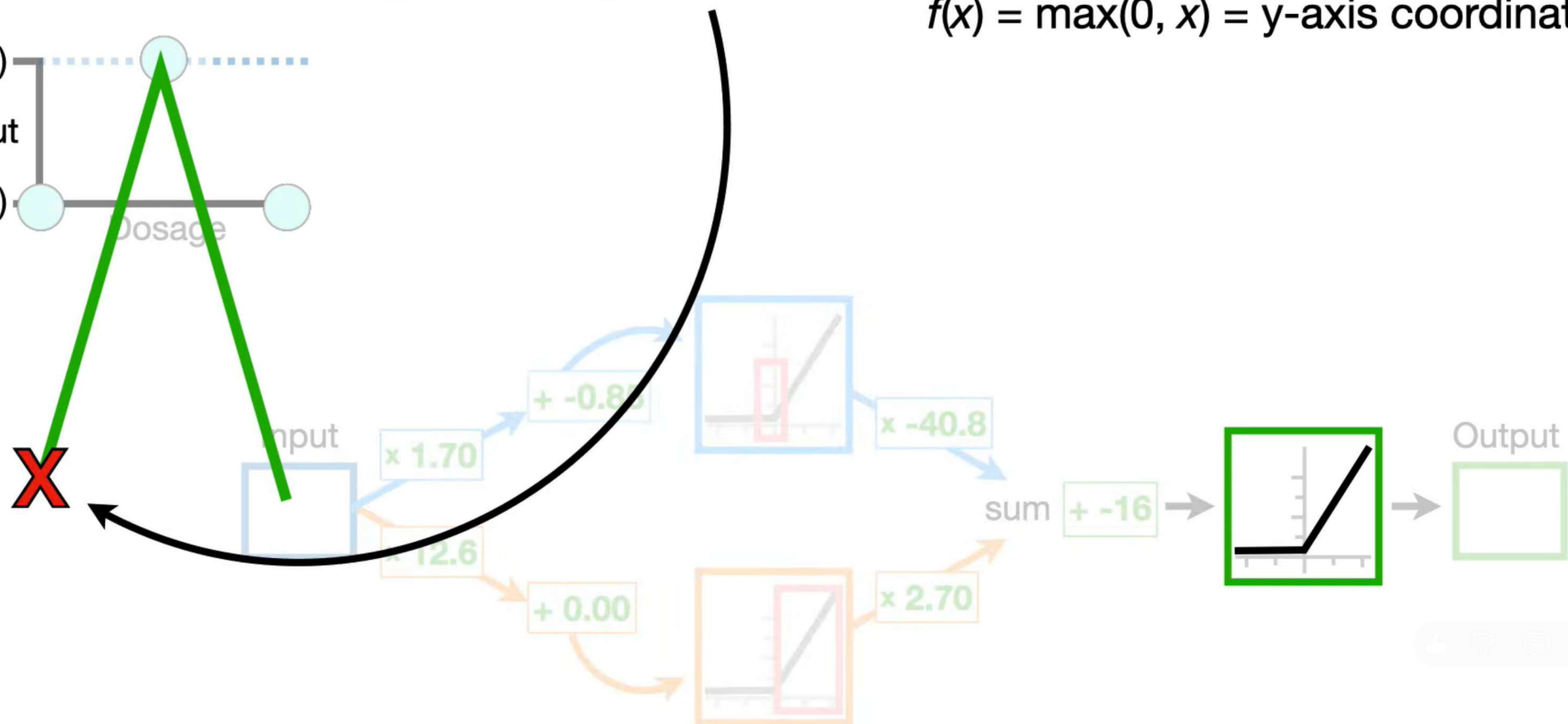
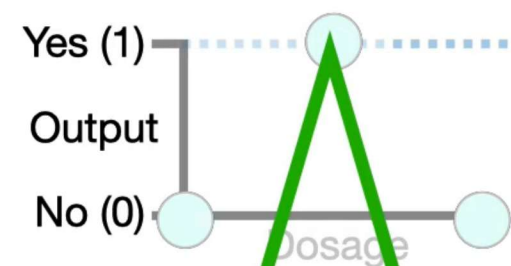
$$f(x) = \max(0, x) = \text{y-axis coordinate}$$





For example, the y-axis coordinate for this point on the **green wedge** is **-16...**

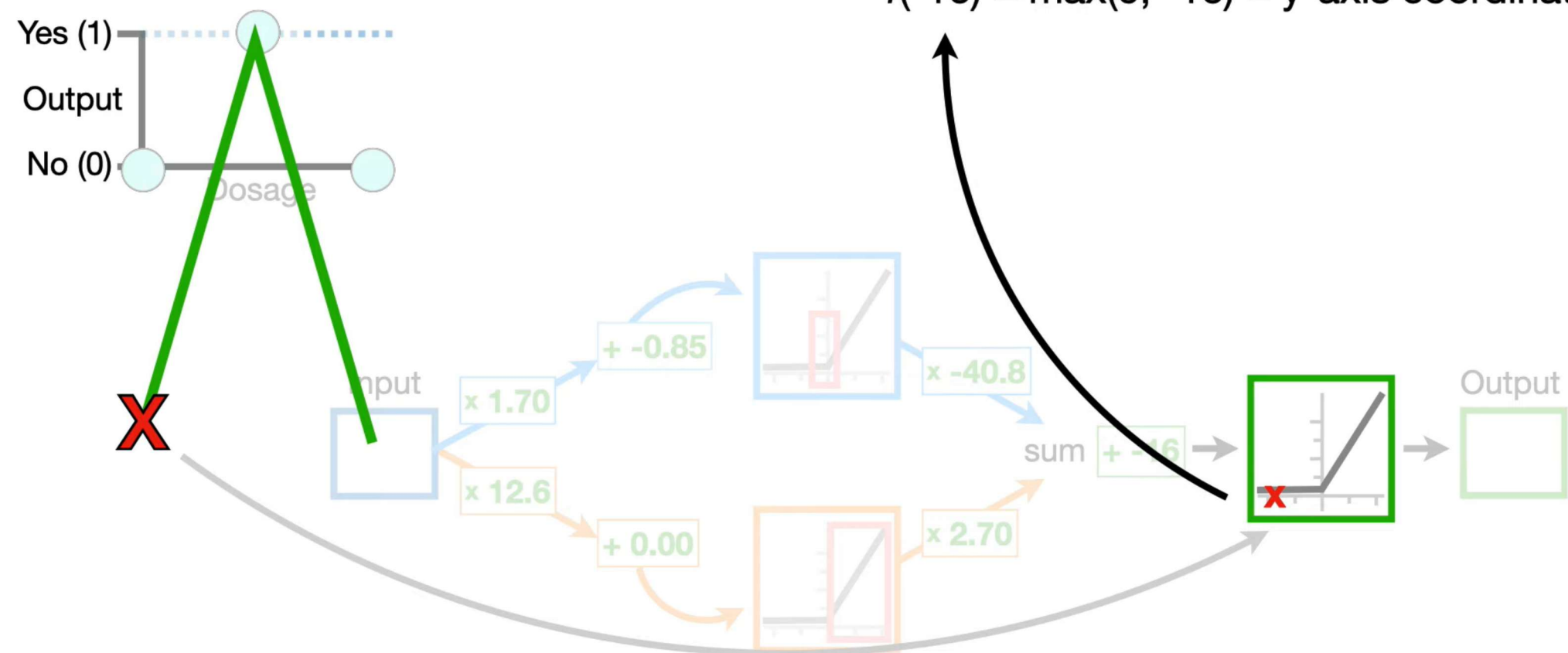
$$f(x) = \max(0, x) = \text{y-axis coordinate}$$





...and when we plug that
into the **ReLU** activation
function, we get...

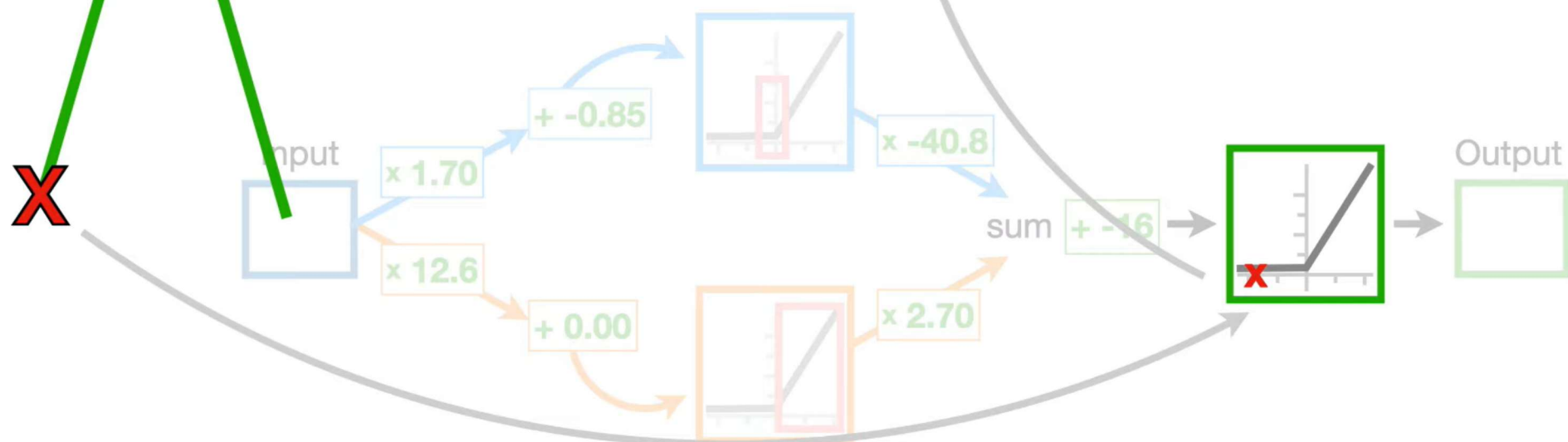
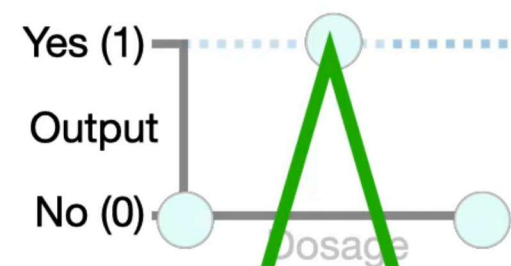
$$f(-16) = \max(0, -16) = \text{y-axis coordinate}$$





...0, because $0 > -16$.

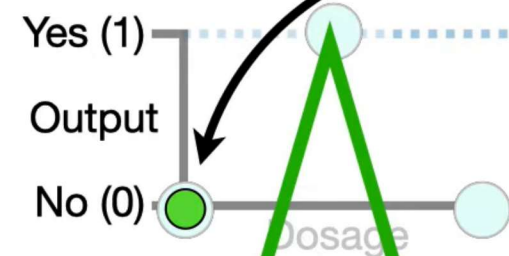
$$f(-16) = \max(0, -16) = 0$$





And **0** corresponds to
this **green dot**.

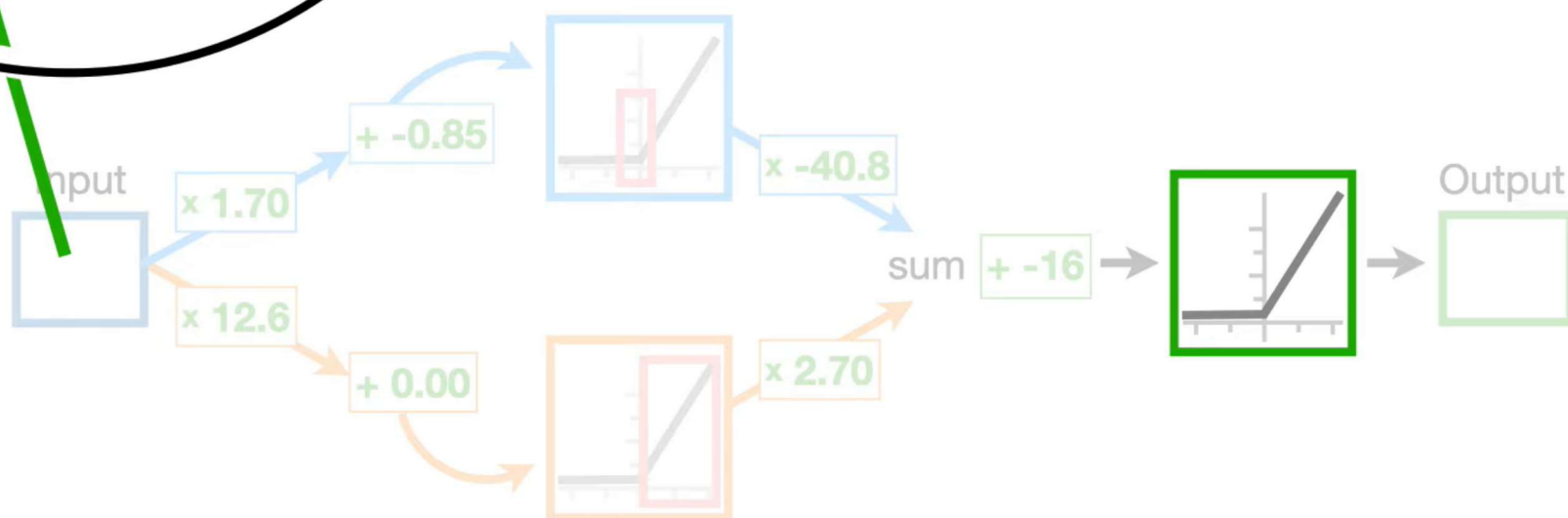
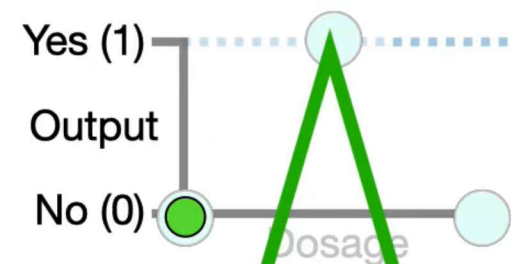
$$f(-16) = \max(0, -16) = 0$$





Likewise, the y-axis coordinate for this point on the **green wedge** is **-9.2...**

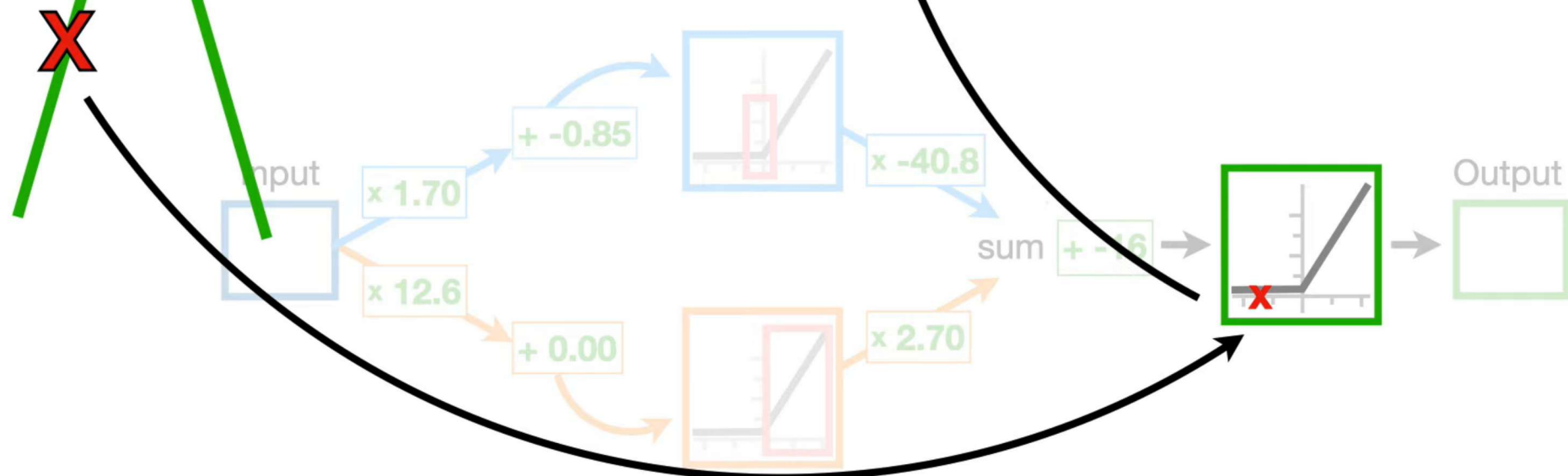
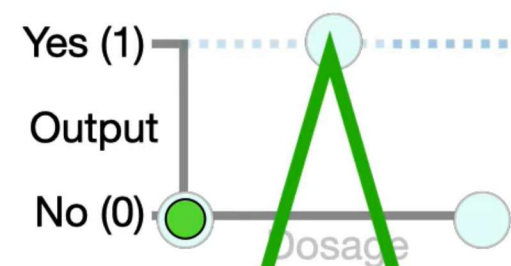
$$f(x) = \max(0, x) = \text{y-axis coordinate}$$





...and, just like before,
when we plug that into the
ReLU activation function,
we get...

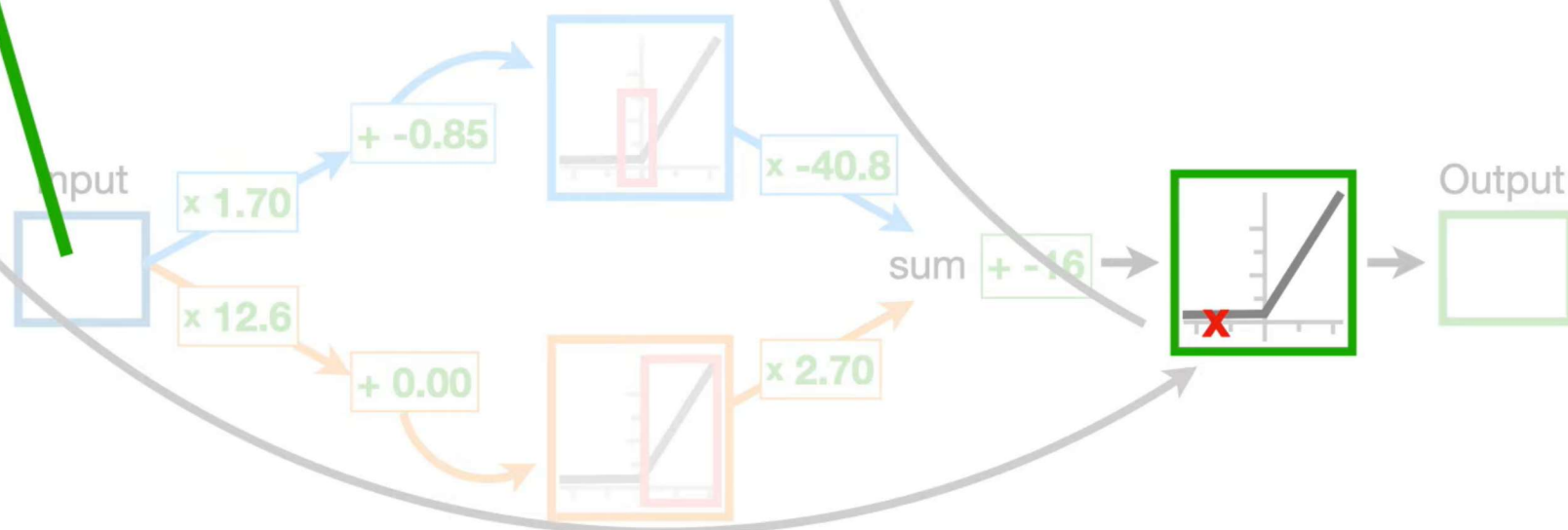
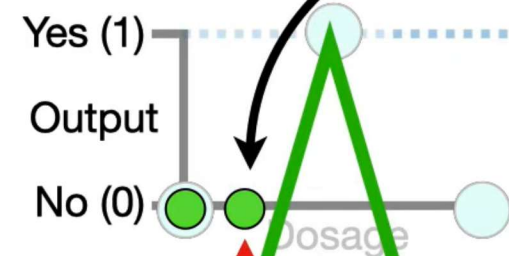
$$f(-9.2) = \max(0, -9.2) = \text{y-axis coordinate}$$





And **0** corresponds to
this **green dot**.

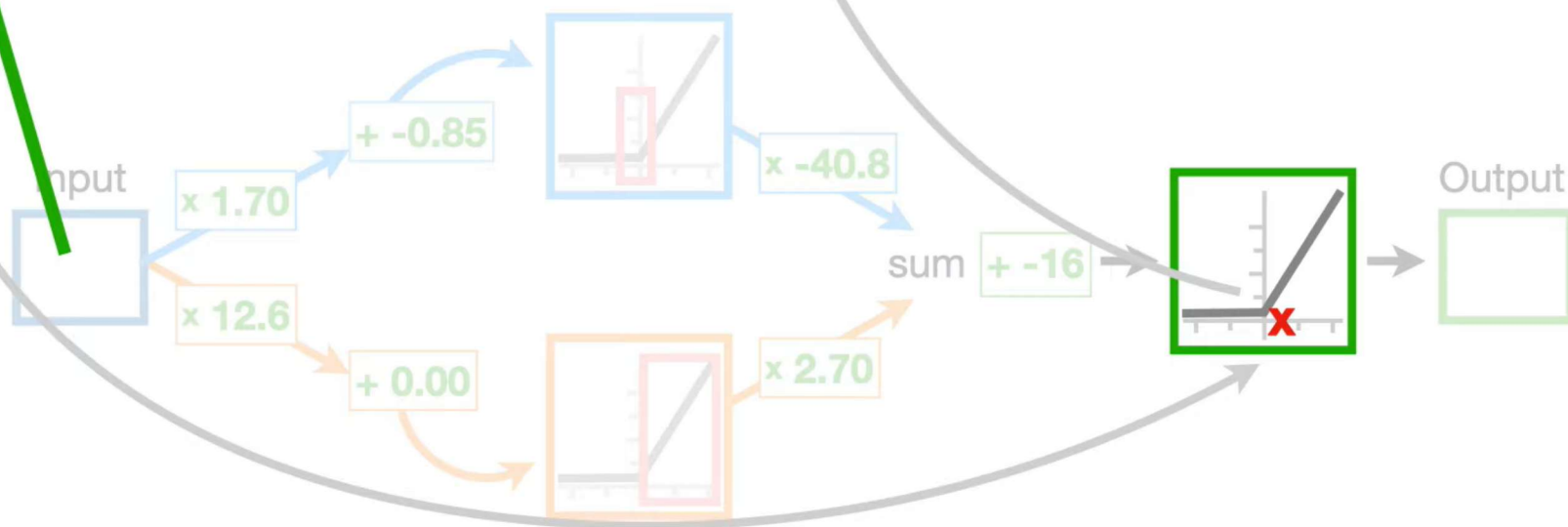
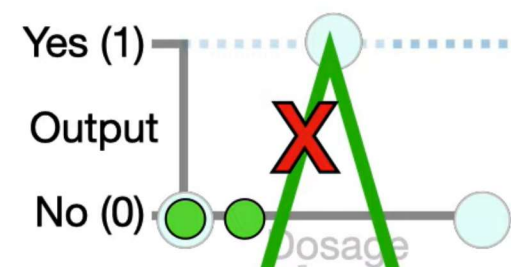
$$f(-9.2) = \max(0, -9.2) = 0$$





...we get **0.49**.

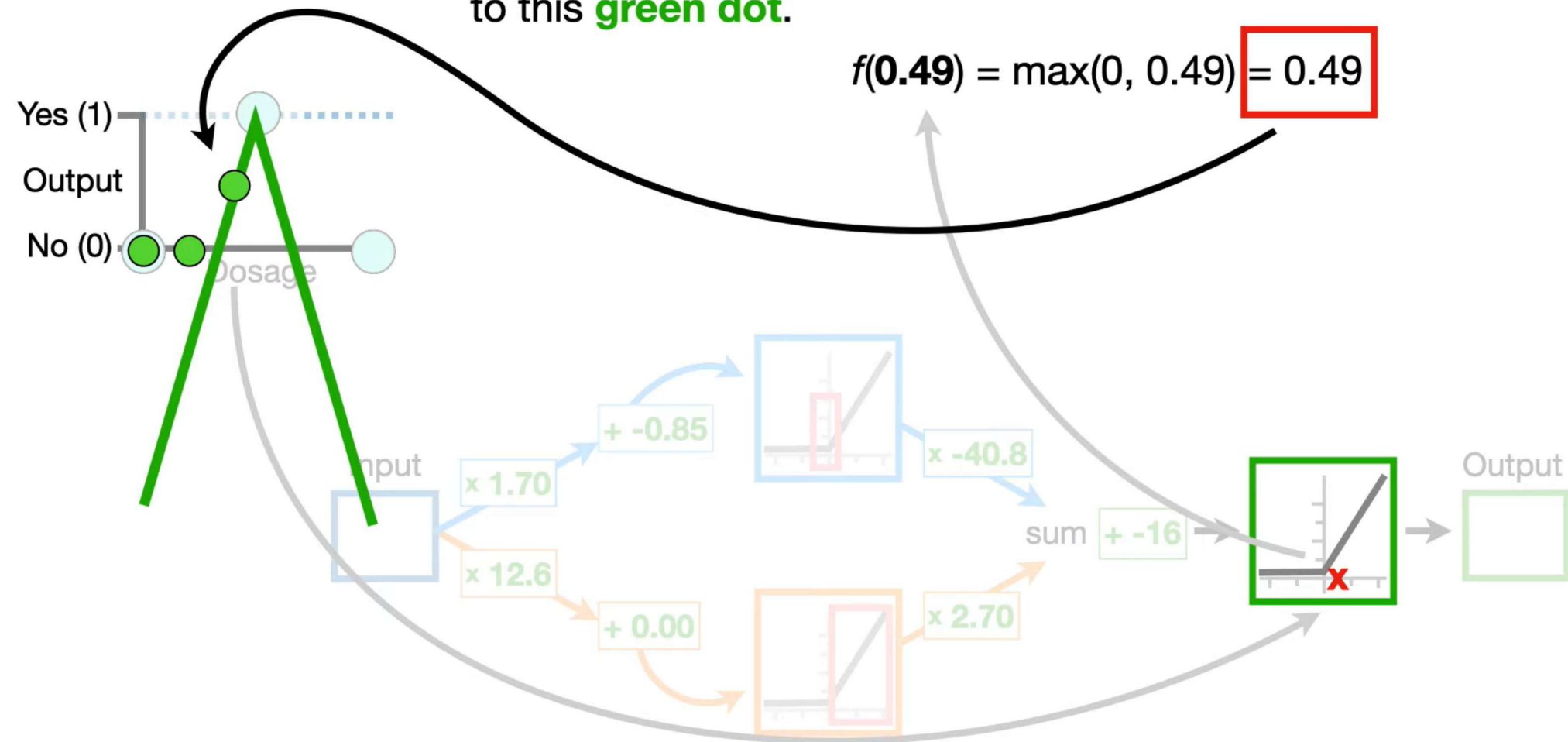
$$f(0.49) = \max(0, 0.49) = 0.49$$





And **0.49** corresponds
to this **green dot**.

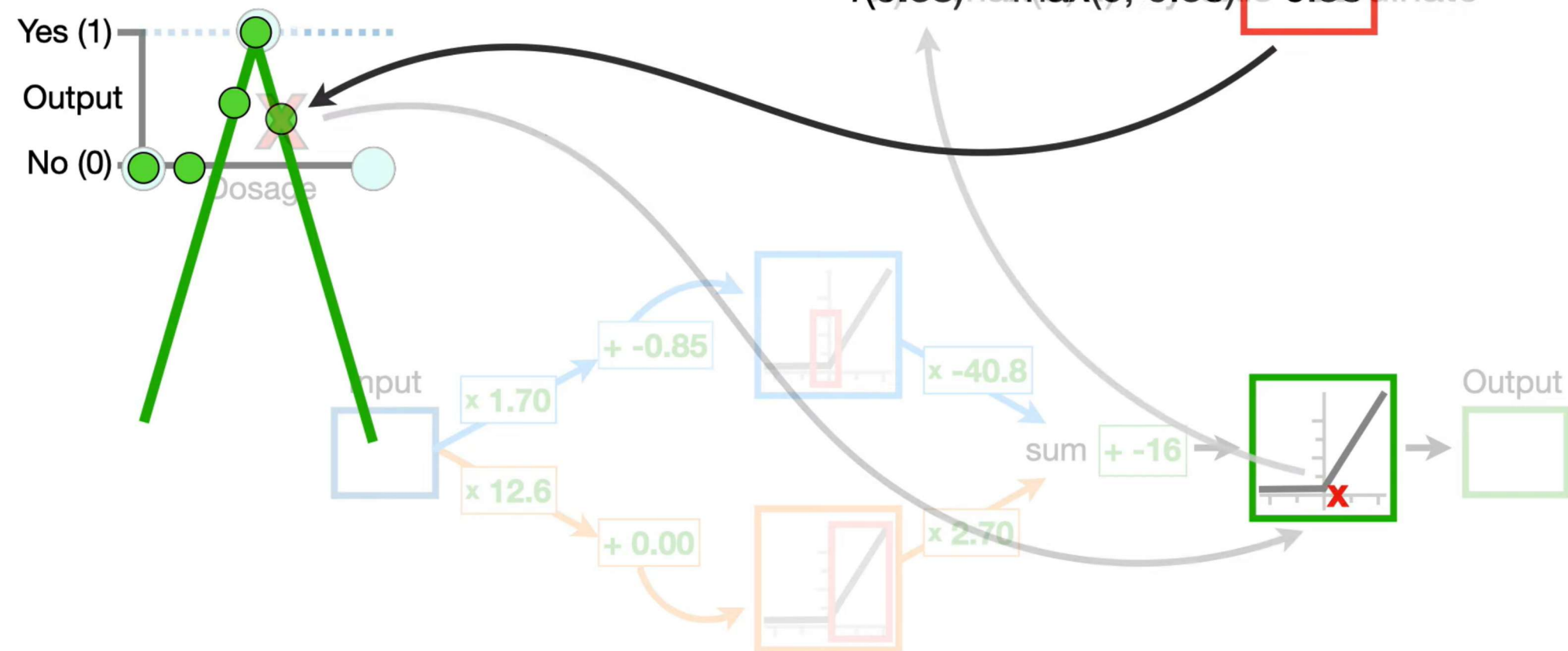
$$f(0.49) = \max(0, 0.49) = 0.49$$





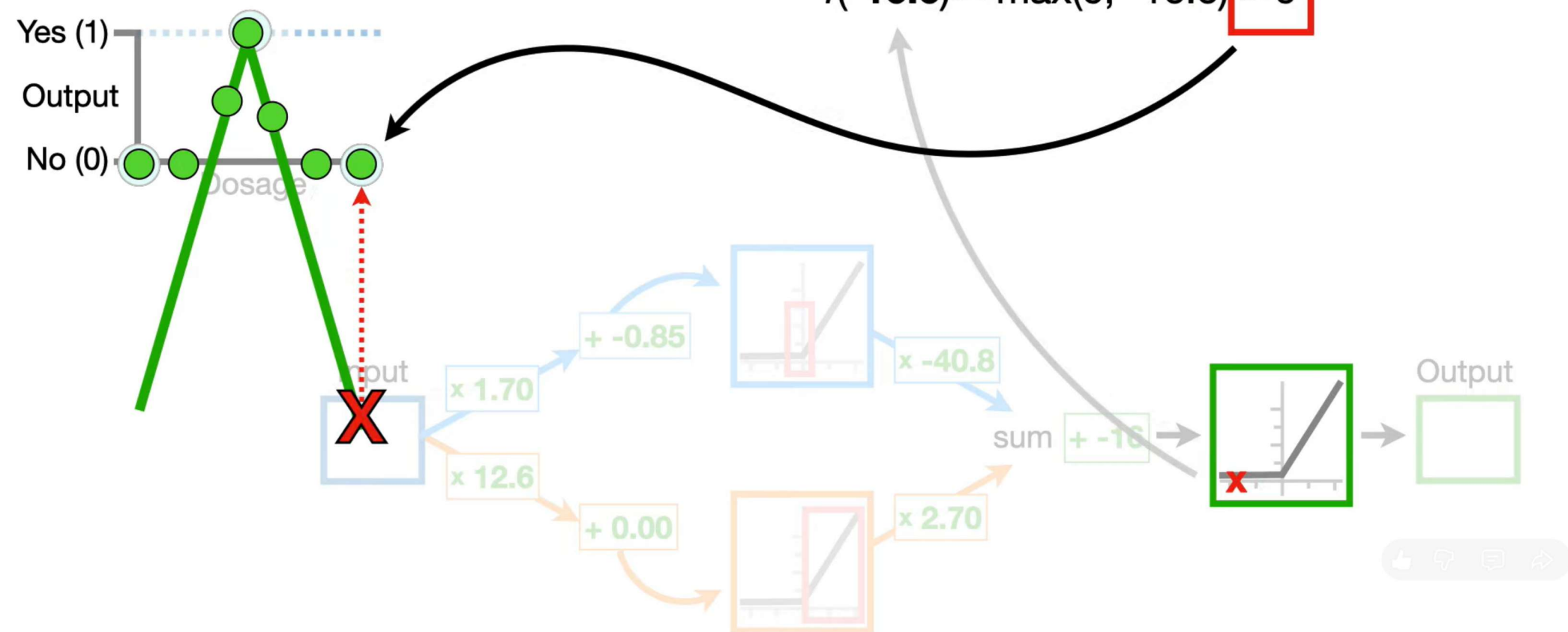
Likewise, the remaining
positive y-axis coordinates
stay the same...

$$f(0.38) = \max(0, 0.38) = 0.38$$



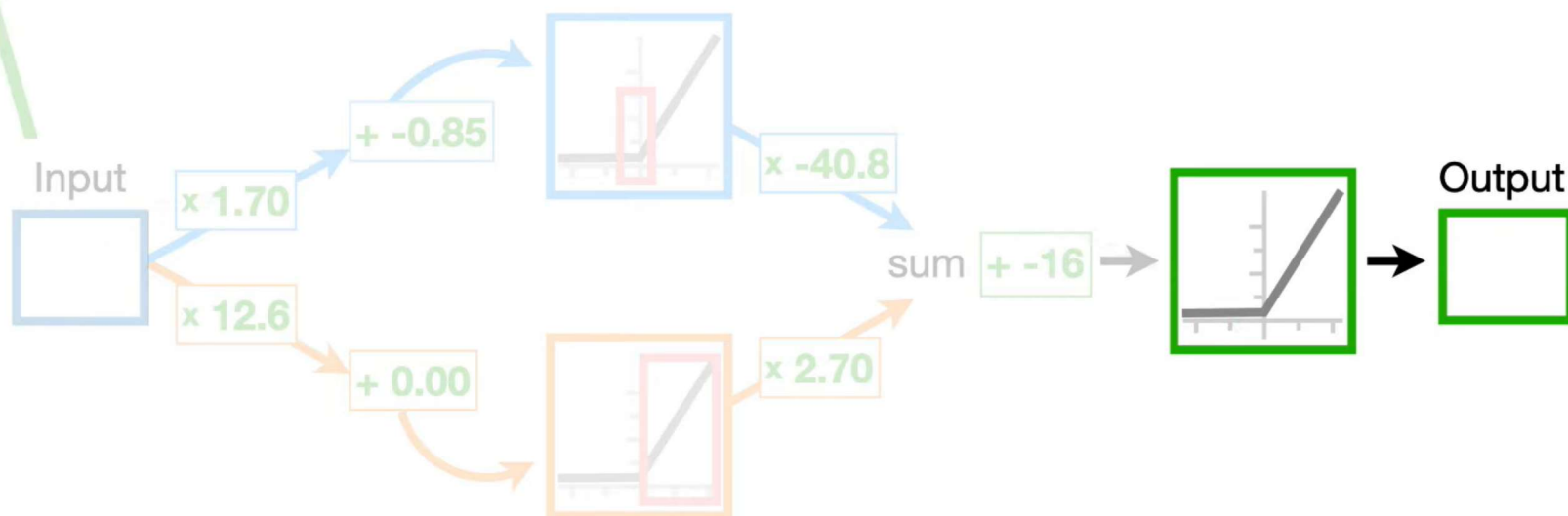
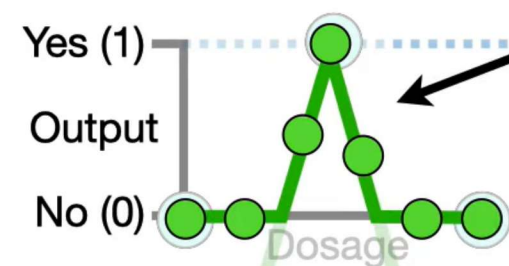
...and the negative values are set to 0.

$$f(-16.6) = \max(0, -16.6) = 0$$



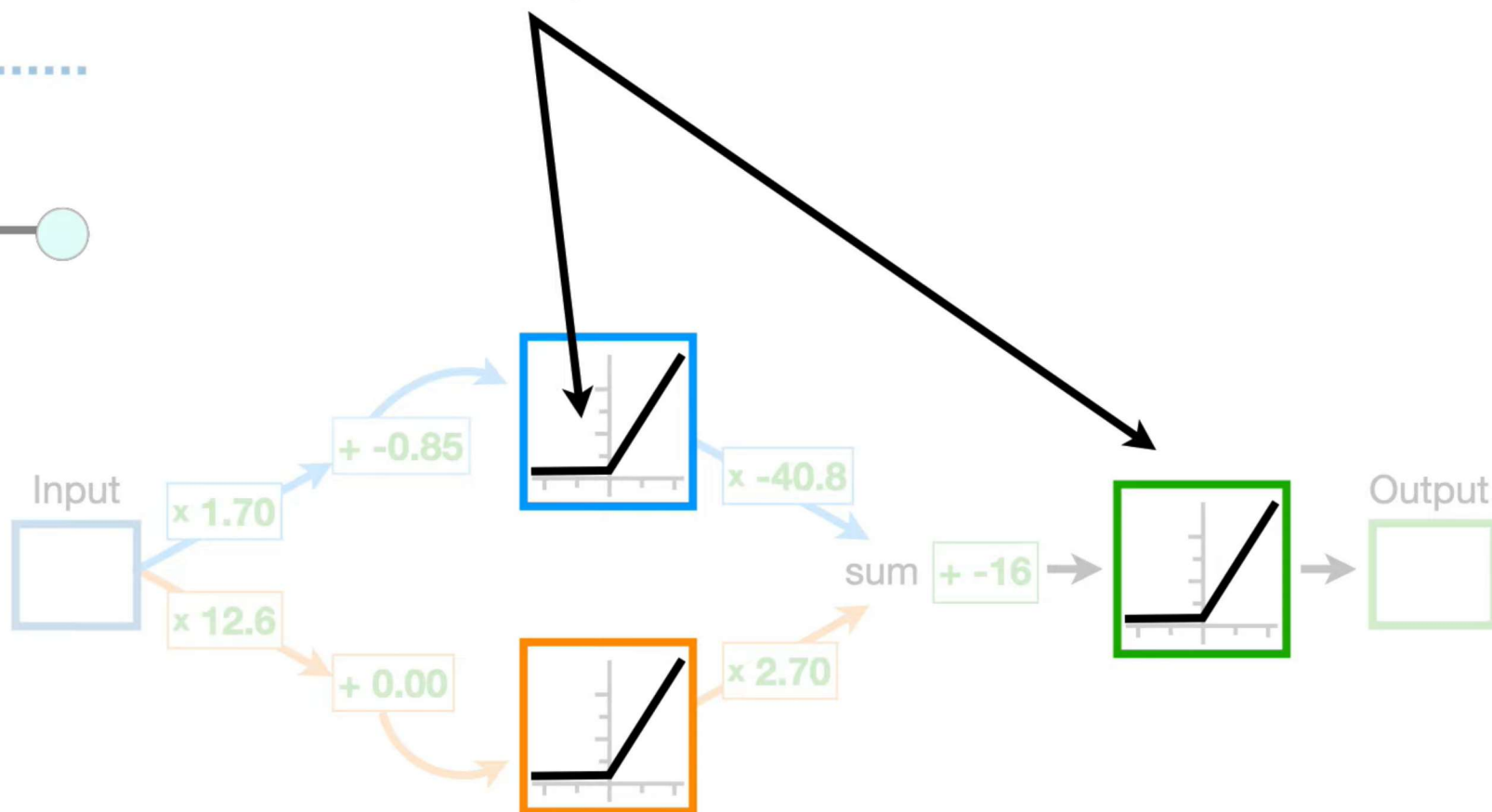
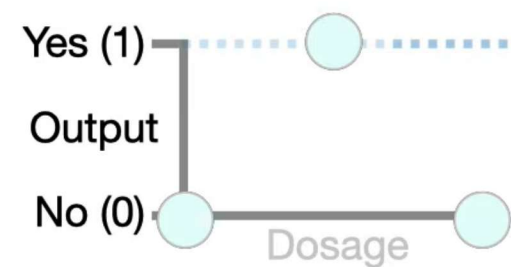


And at long last, we end up with this **green pointy thing**.





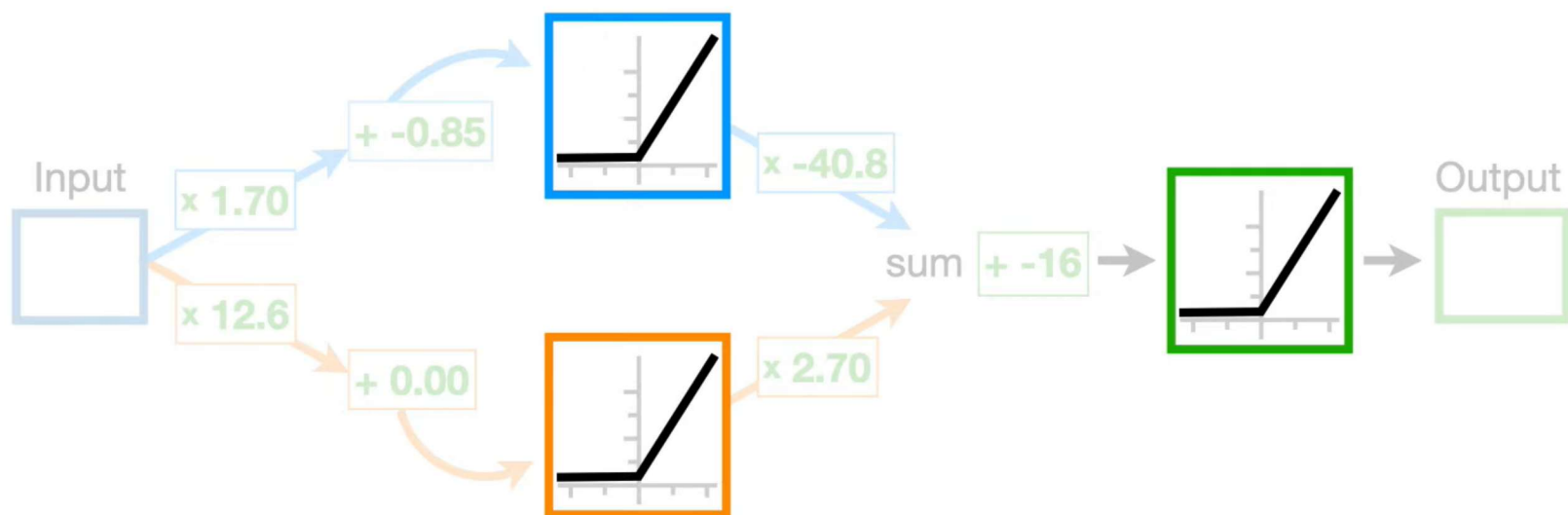
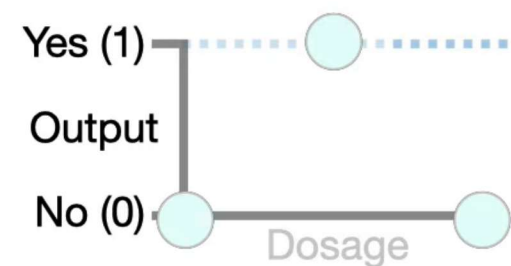
Thus, the **ReLU** activation function may seem weird, because it is not curvy...





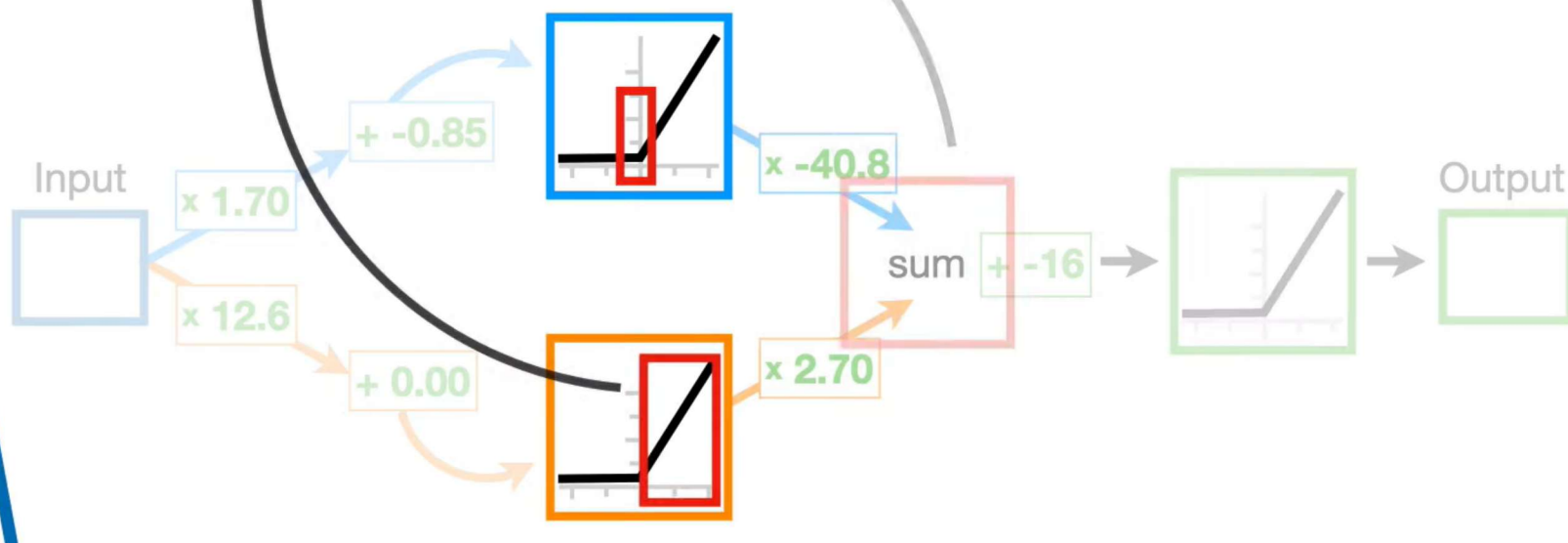
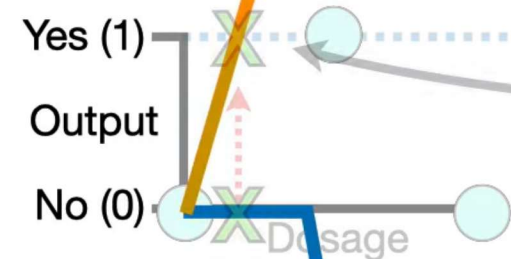
...and the equation is really simple.

$$f(x) = \max(0, x) = \text{y-axis coordinate}$$



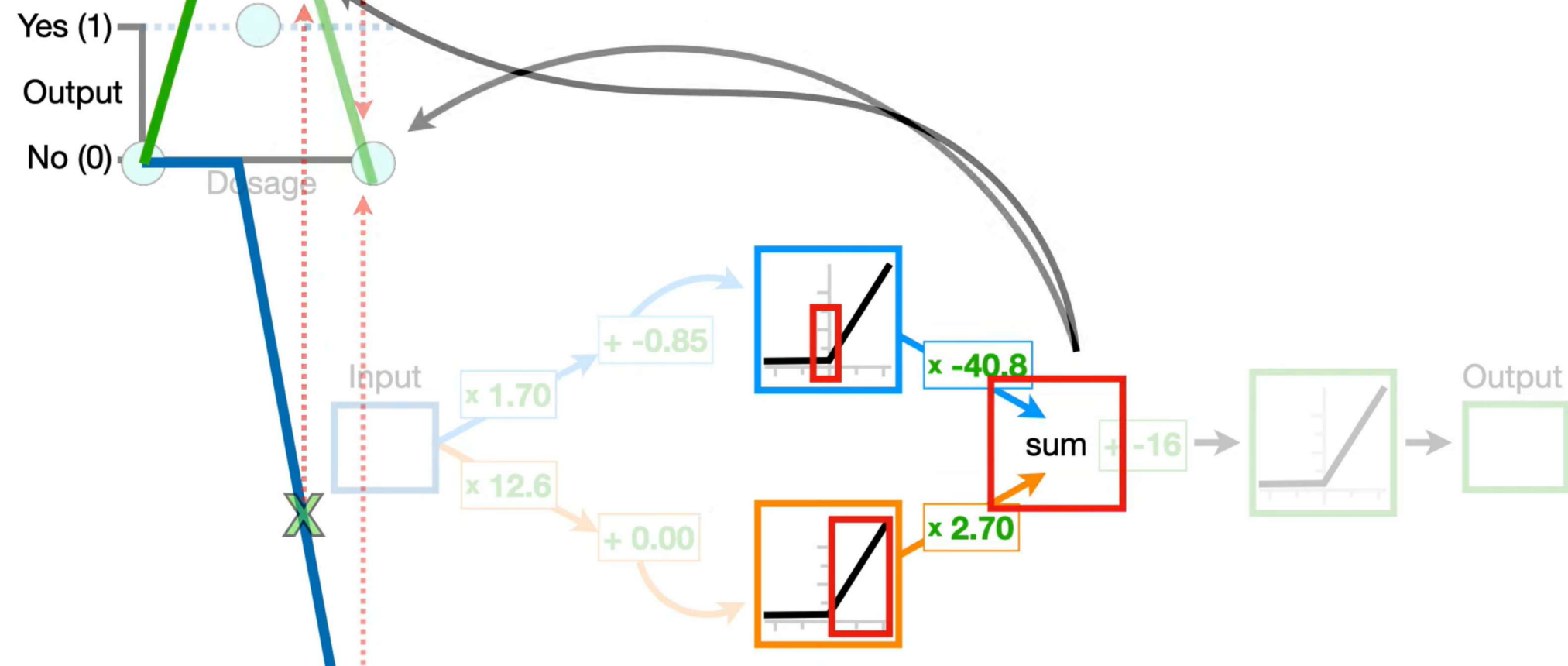


But just like for any other activation function, the **Weights** and **Biases** on the connections slice them, flip them and stretch them into new shapes...



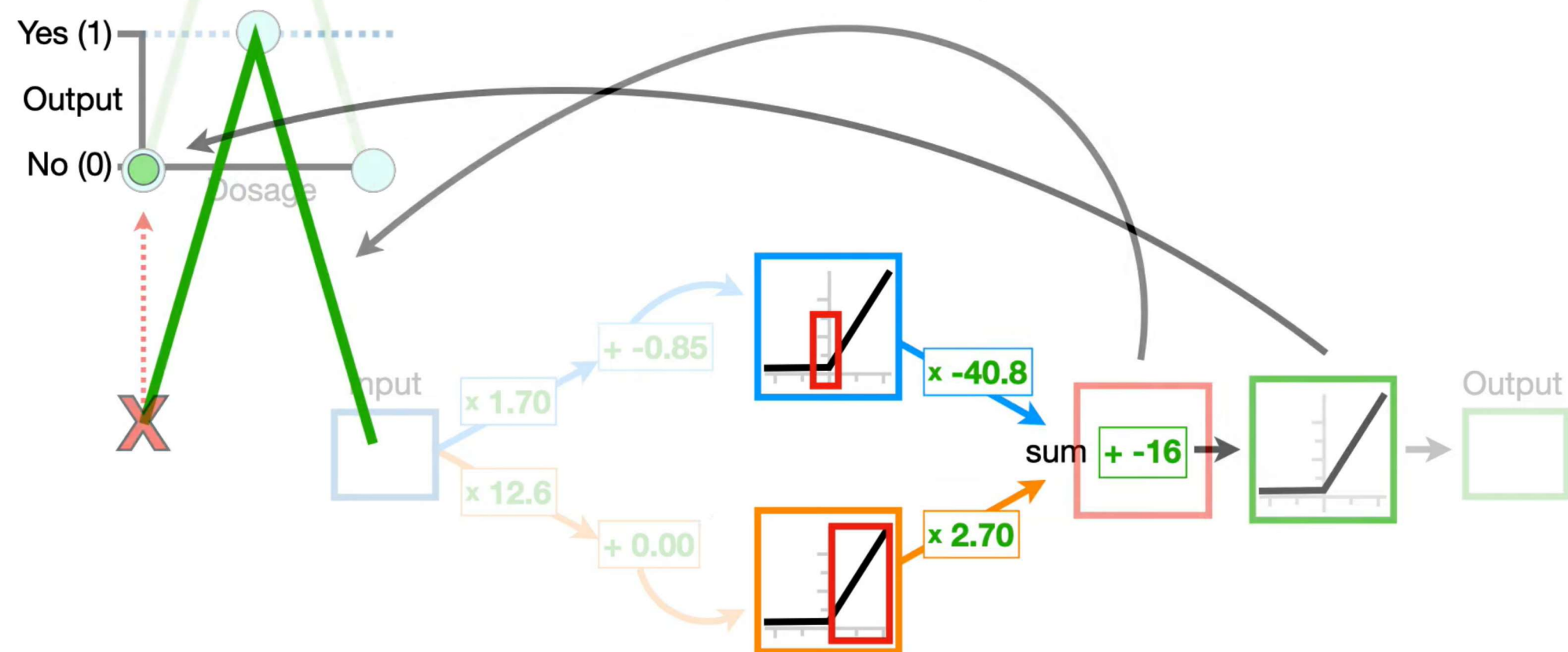


But just like for any other activation function, the **Weights** and **Biases** on the connections slice them, flip them and stretch them into new shapes...



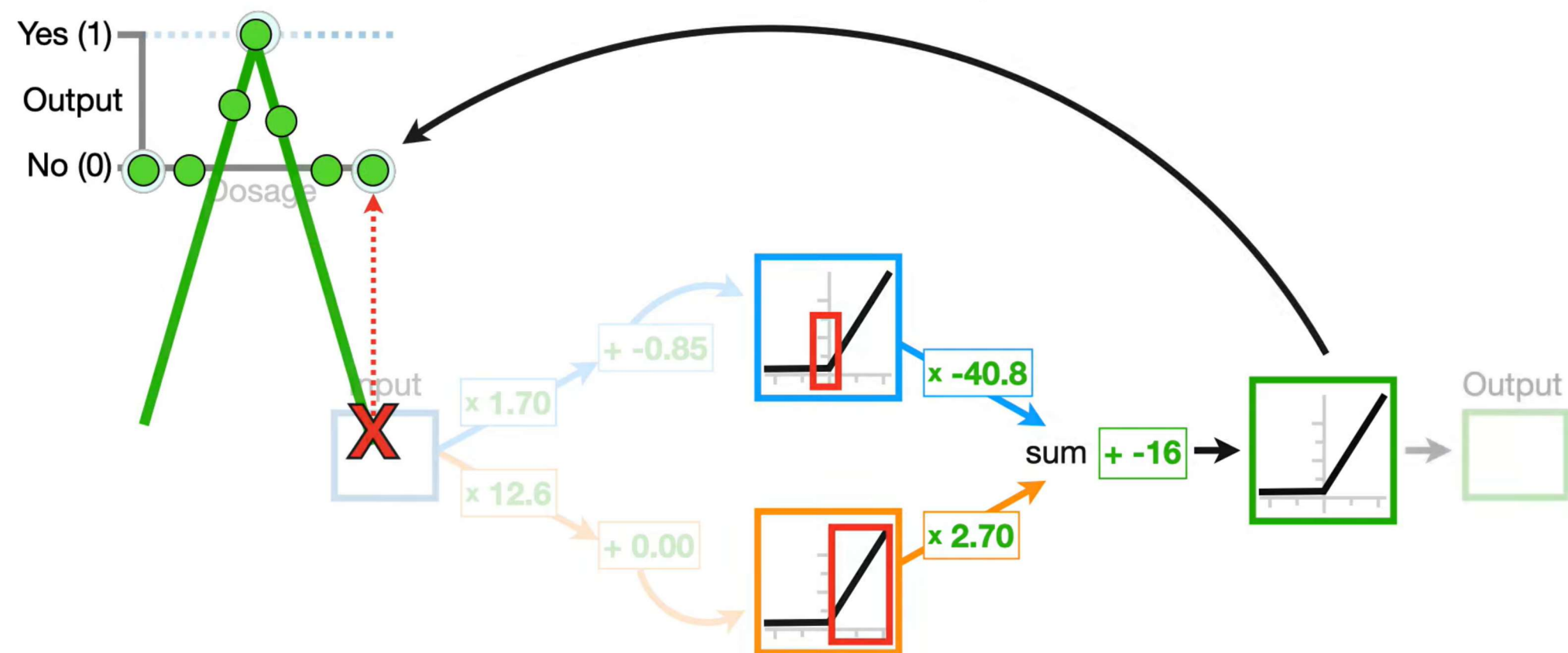


But just like for any other activation function, the **Weights** and **Biases** on the connections slice them, flip them and stretch them into new shapes...



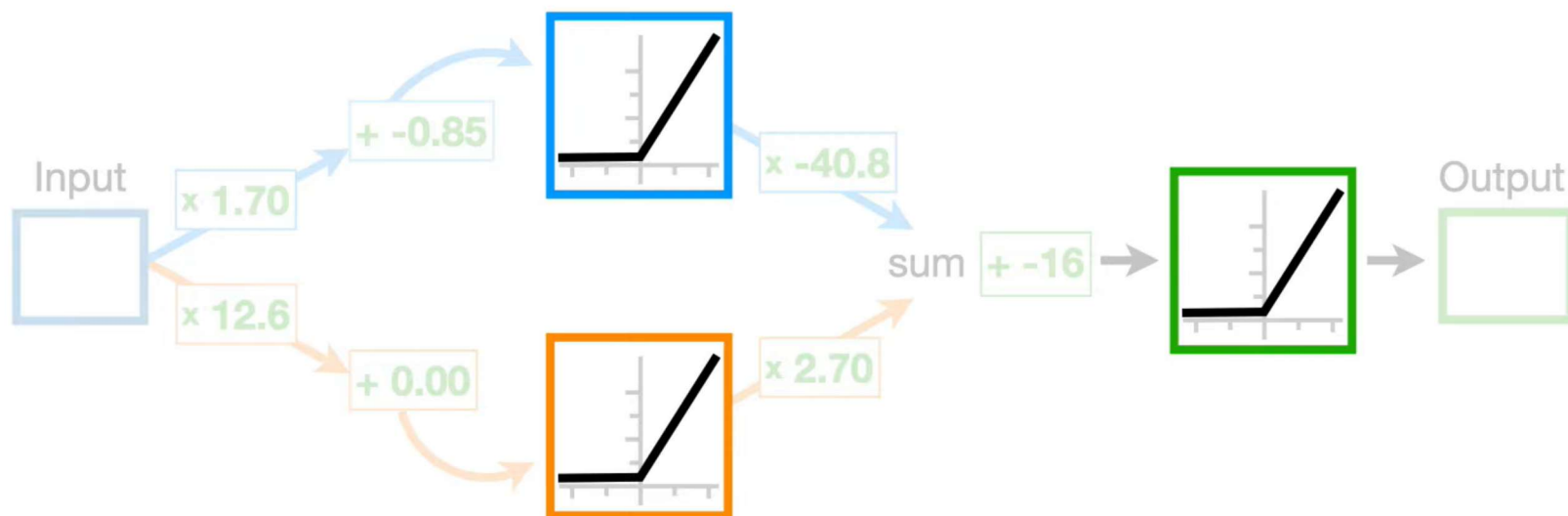
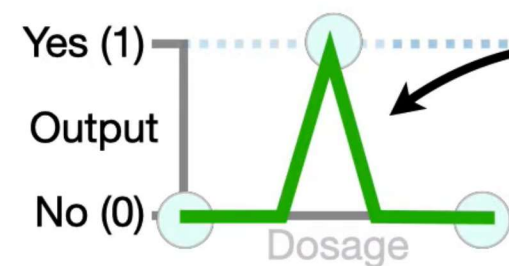


But just like for any other activation function, the **Weights** and **Biases** on the connections slice them, flip them and stretch them into new shapes...





...which are added together to get an entirely new shape that fits the data.





How to handle negative dataset?

✓ 1. Normalize/standardize your input

Negative values are fine — the model will shift them using weights.

But you should scale the data using:

- StandardScaler (mean=0, std=1)
- Min–Max scaling to [-1,1] or [0,1]

✓ 2. Use LeakyReLU instead of ReLU

Leaky ReLU allows a small slope for negative values:

$$\text{LeakyReLU}(x) = \begin{cases} x & x > 0 \\ 0.01x & x \leq 0 \end{cases}$$

So negative inputs do NOT become zero.

✓ 3. Use ELU or GELU

These activation functions keep small negative values and avoid dead neurons.

Example Visualization

If dataset is like:

[-9, -7, -3, -1, 0, 1, 2]

ReLU output:

[0, 0, 0, 0, 0, 1, 2]

Most information is lost.

So, using **ReLU directly on these raw inputs is NOT a good idea**, because all negative numbers will become **0**, and most information will be lost.

Here's what you **should do instead**, depending on your goal.

Option 1: Normalize / Standardize the data

This is the MOST common solution in ML.

Standardization (Recommended)

Transforms data to mean = 0 and std = 1:

Example result might be:

[-1.5, -1.2, -0.6, -0.2, 0.0, 0.3, 0.5]

Now negative values are small, not huge.

ReLU can handle this because after multiplying with weights + bias, they can become positive.

Option 2: Use Leaky ReLU instead of ReLU

Leaky ReLU keeps negative inputs:

$$x \leq 0 \rightarrow 0.01x$$

So your dataset becomes:

[-0.09, -0.07, -0.03, -0.01, 0, 1, 2]

Meaning:

 small negative slope

- ✓ no information lost
 - ✓ no dead neurons
-

✓ Option 3: Shift the data

Add a constant to make all data positive.

Example:

shift = 10

new data = [

1, 3, 7, 9, 10, 11, 12

]

Now ReLU behaves normally.

But this is less common than normalization.

✓ Option 4: Use another activation (ELU, GELU, SELU)

For example, ELU keeps negative values but shrinks them smoothly.

This avoids ReLU's "dead neuron" issue.

🧠 Best practice

For this dataset:

1. Always normalize or standardize first

👉 This is the correct ML practice.

2. Use LeakyReLU instead of ReLU (if you want to keep negative info)

3. Do NOT directly feed raw [-9, -7, -3, -1...] into ReLU

It will destroy information.

What happens if my dataset is mostly negative and use only ReLU?

✔ Case 1: Input to the model is always negative

If you directly feed the raw dataset and it is **all negative**, then:

- ReLU will turn all inputs into 0
- The first layer receives only zeros
- The network **cannot learn anything**
- This is called a **dead layer**

✔ Case 2: Activations become negative inside the network

Even if the raw dataset has negative values, **weights + bias** in the network will change the values before applying ReLU:

$$z = Wx + b$$

This can make the values positive again.

So ReLU only kills negative activations **after** the linear transformation.

⚠ Why negative values can be a problem with ReLU

ReLU has a famous problem called:

Dying ReLU Problem

If **too many values are negative**, they all become 0, and gradient becomes 0:

$$\text{ReLU}'(x) = \begin{cases} 1 & \text{if } x > 0, \\ 0 & \text{if } x \leq 0. \end{cases}$$

A neuron that always outputs 0 will **never update again**.
